

Guía de uso

PROJECT AK

Desarrollado por:

Torres Cardona Alan

Club TECLA

Proyecto iniciado:

10 de Diciembre de 2025

ÍNDICE

Las páginas se calculan automáticamente al abrir el documento. Si algún enlace no funciona, haz clic dentro del índice y pulsa F9 para actualizarlo.

1. Introducción a Project AK	1
2. ¿Cómo iniciar?	1
3. Configuración y conexión al Engine	1
4. Configuración de juego	1
5. ¿Cómo inicializar una escena?	1
6. ¿Qué es un Objeto?	1
7. ¿Qué podemos hacer con un Objeto?	1
7.1 Componentes.....	1
8. Arquitectura general	1
8.1 Las cuatro capas.....	1
8.2 El arranque de la aplicación.....	1
8.3 El bucle principal, fotograma a fotograma	1
9. Servicios: el acceso global.....	1
10. Escenas y el gestor de escenas.....	1
10.1 Qué es una escena.....	1
10.2 Registrar y cambiar de escena.....	1
10.3 TileMaps dentro de una escena.....	1
11. Objetos y el ObjectPool.....	1
11.1 Añadir, buscar y quitar	1
11.2 Cómo se ordenan los objetos al dibujar.....	1
11.3 Capas de dibujado	1
12. Componentes en profundidad	1
12.1 Cómo funcionan por dentro	1
12.2 Añadir, consultar y comprobar.....	1
12.3 Catálogo de componentes.....	1
12.4 Los tres constructores de ISprite	1
13. Entrada: teclado, ratón y mando.....	1
13.1 El ciclo de los eventos	1
13.2 Referencia de la API.....	1
13.3 Distinguir pulsación de mantenimiento	1
14. El renderizador y el lote de dibujado.....	1
14.1 El Batcher	1
14.2 Primitivas de dibujado.....	1
14.3 El objeto Render	1
14.4 El ciclo de dibujado de un objeto	1
15. La cámara 2D.....	1
15.1 Crearla y conectarla.....	1
15.2 API de Camera2D.....	1
15.3 Seguimiento suave con la rueda.....	1

16. Recursos: texturas, atlas y el array de texturas.....	1
16.1 Cargando recursos	1
16.2 Qué contiene un atlas.....	1
16.3 Coordenadas de textura (UV).....	1
17. Shaders y materiales.....	1
17.1 Cargar un shader	1
17.2 Asignar un material a un objeto	1
17.3 Anatomía de un shader.....	1
18. TileMaps	1
18.1 Cargar un mapa de Tiled	1
18.2 Orientación del mapa.....	1
18.3 API de TileMap	1
18.4 Dibujar un subrange de capas.....	1
19. Matemáticas del juego: Vector2, Color y geometría.....	1
19.1 Vector2.....	1
19.2 Color	1
19.3 Geometría.....	1
20. Texto y fuentes	1
20.1 Horneear una fuente	1
20.2 Cargar y registrar una fuente.....	1
20.3 La clase Font	1
20.4 Escribir texto	1
21. Interfaz de usuario.....	1
21.1 Los tres controles.....	1
21.2 Crear un botón.....	1
21.3 Etiquetas y campos de texto.....	1
22. Audio: música y efectos	1
22.1 Música.....	1
22.2 Efectos de sonido	1
22.3 Controlar el volumen con el teclado.....	1
23. Diálogos y CSV.....	1
23.1 Cargar una tabla.....	1
23.2 Leer un diálogo	1
23.3 El componente IDialog.....	1
24. Colisiones y físicas.....	1
24.1 Las cajas de colisión.....	1
24.2 Funciones de colisión	1
25. El tiempo: dt y el reloj.....	1
25.1 Cómo se mide el tiempo	1
25.2 Mover usando dt.....	1
26. Rutas y configuración.....	1
26.1 ENG::Path.....	1
26.2 Qué se copia al compilar	1
26.3 El archivo de configuración.....	1
26.4 Guardar el estado de la ventana.....	1
27. Registro de mensajes (logging).....	1
27.1 Las cuatro macros	1

28. Guardar y cargar el mundo.....	1
28.1 La API.....	1
28.2 Guardar y recuperar un objeto.....	1
28.3 Serialización con nlohmann/json.....	1
29. Mapa del proyecto.....	1
29.1 Raíz del repositorio.....	1
29.2 Dentro de src/.....	1
29.3 Dentro de src/Engine/.....	1
29.4 Dentro de src/Game/.....	1
30. Referencia rápida: "¿cómo hago X?".....	1
31. Errores comunes.....	1
31.1 Errores de compilación.....	1
El archivo nuevo no se compila.....	1
Los includes no se resuelven.....	1
El atributo se repite.....	1
Falta la implementación de una biblioteca.....	1
31.2 Errores de ejecución.....	1
Pantalla en negro.....	1
El mapa sale desplazado.....	1
Se ve la textura equivocada.....	1
Las teclas no responden.....	1
El cambio de escena no ocurre.....	1
El juego va más rápido o más lento según el equipo.....	1
Los objetos se dibujan en orden raro.....	1
Errores de compilación y depuración.....	1
31.3 Errores de diseño.....	1
32. Glosario.....	1

1. Introducción a Project AK

Como primera instancia, el motor gráfico que eh desarrollado sigue en fase de desarrollo Alpha, existen muchos bugs o falta de contenidos que con el paso del tiempo esta guía ira solventando y anexando.

Este proyecto inicio un 10 de diciembre de 2025, en una clase de Videojuegos, con la idea de ser sencillo, fácil de entender, no preocuparte por mucho, ligero, minimalista y propio.

Para introducir a Project AK, no hay nada mejor que introducir como es que funciona el Engine.

Se tiene que construir todo, desde las escenas, hasta lo que debe de contener cada escena, el engine cuenta con todos los recursos que hasta ahorita fueron contemplados para iniciar un proyecto (Esto SI ira aumentando con el paso del desarrollo).

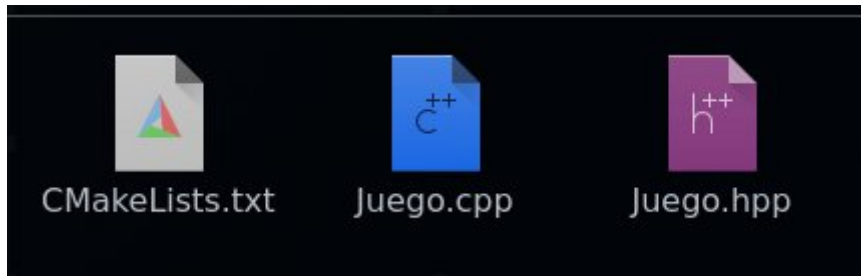
2. ¿Cómo iniciar?

Primeramente explicare como iniciar una escena. La estructura de tu juego es independiente del motor gráfico, puedes crear un directorio a la par de la carpeta Engine/ llamada Juego/ o como tu lo quieras estructurar, como recomendación siempre separa todo por carpetas y junta archivos en sus correspondientes.

Iniciaremos por crear una carpeta llamada Juego/ y esta sera nuestra raíz de nuestro juego.



Y dentro de la carpeta Juego/, contendrá los archivos Juego.cpp y Juego.hpp que sera la conexión del Engine->Juego. (Actualmente estos archivos están en blanco)



3. Configuración y conexión al Engine

Una vez construida la conexión (Sin código) del Engine->Juego, debemos configurar los archivos CMake que sera la conexión para poder buildear nuestro proyecto.

En el directorio Juego/ crearemos un archivo llamado CMakeLists.txt, el cual sera la conexión de nuestro Juego con el proyecto en su totalidad. Escribiendo lo siguiente:

```
target_sources(Project_AK PRIVATE Juego.cpp)
```

Una vez teniendo esta conexión del Juego->Engine con CMake, podemos ahora buildear y descargar dependencias que ocupe el Engine para continuar. En la raíz de todo el proyecto crearemos una carpeta llamada build/, en la cual ingresaremos y escribiremos el siguiente comando desde la carpeta build:

cmake -G"Ninja Multi – Config" ..

Descargara y configurara todo el proyecto, cabe mencionar que cada vez que se agreguen carpetas nuevas y archivos nuevos al proyecto, se debe de ingresar a la carpeta build/ y re-generar el comando, para enlazar todo correctamente.

4. Configuración de juego

Una vez teniendo la conexión previa, ahora si podemos agregar el código que enlazara el Motor gráfico con el juego.

Este sera el código para tener la base del juego inicializada:

HPP:

```

/// | ----- |
#pragma once
/// | ----- |
#include "Engine/Layer/GameLayer.hpp"
/// | ----- |

namespace APP
{
    class Game: public ENG::GameLayer
    {
    public:
        /// Explicit constructor, we dont need conversions
        explicit Game();
        /// Defatult destructor #Inheritance
        ~Game()override = default;
        /// Init all stuf for the game. #Inheritance
        void OnInit(void) override;
        /// First inputs to take, #Inheritance
        void OnInputs(float dt ) override;
        /// Destroy all stuf of the game, #Inheritance
        void OnDestroy(void) override;
        /// Validated if the game still running, #Inheritance
        bool IsRunning(void) override;
        /// Update all the Game, #Inheritance.
        void OnUpdate(float dt) override;
        /// UpdateFixed (For physics), #Inheritance.
        void OnUpdateFixed(float dt) override;
        /// Render all stuf of the Game, #Inheritance.
        void OnRender(float dt) override;
    private:
        /// Boolean, if the game is running.
        bool m_gameRunning = false;
    };
}

```

Tomamos del Engine la capa de Layer/GameLayer.hpp, que sera la herencia que interpreta el Motor gráfico, de aquí solo es conexión con la escena actual y las escenas que manejemos, tomar recursos del motor gráfico y todo lo demás seria casi “Automático”.

Ahora continuaremos con el CPP:

```

/// | ----- |
#include "Juego.hpp"
/// | ----- |
#include "Engine/Utils/Log.hpp"
#include "Engine/Services/ScenesManager.hpp"
#include "Engine/Services/Services.hpp"
#include "Engine/Render/Render.hpp"
/// | ----- |
#include <string>
/// | ----- |

namespace APP
{
    Game::Game()
    {
        m_gameRunning = true;
    }

    void Game::OnInit(void)
    {
        LOG_INFO(message: " | << Game application Init Succesfully");
        m_currentScene = ENG::Services::Scenes().GetCurrent();
        m_currentScene->Init();
    }

    void Game::OnDestroy(void)
    {
        LOG_INFO(message: " | << Destroying Game, waiting...");
        m_gameRunning = false;
    }

    bool Game::IsRunning(void)
    {
        return m_gameRunning;
    }

    void Game::OnInputs(float dt)
    {
        m_currentScene->Inputs(dt);
    }
}

```

```
void Game::OnUpdate(float dt)
{
    // while running the current scene
    if(!m_currentScene->IsRunning())
    {
        m_currentScene->Destroy();
        ENG::Services::Scenes().ChangeScene();
        ENG::Services::Scenes().GetCurrent()->Init();
        return;
    }
    m_currentScene->Update(dt);
}

void Game::OnUpdateFixed(float dt)
{
    /// Fixed Update for physics
    m_currentScene->UpdateFixed(dt);
    (void)dt;
}

void Game::OnRender(float dt)
{
    auto& b: Batcher & = ENG::Render::Get().GetBatcher();
    m_currentScene->Render(&b);
}
}
```

La implementación del CPP, realmente solo es tomar la escena actual, renderizar y usar la lógica de la escena que esta en funcionamiento (Actual).

Lo interesante es que podemos consumir, usar o preparar cosas antes de que entre a la escena actual. Comenzaremos en conectar una escena llamada Demo.

5. ¿Cómo inicializar una escena?

Para mantener una estructura limpia crearemos en la carpeta Juego/ una carpeta que se llamara Escenas/. Aquí colocaremos todas las escenas que contendrá el juego.

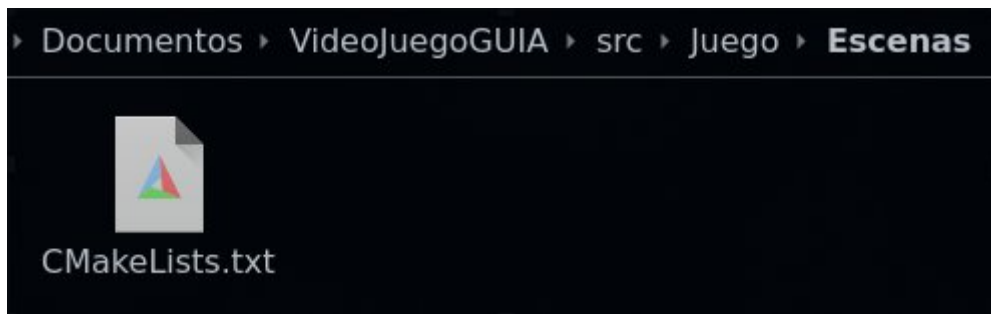


Y amismo actualizamos el CMakeLists.txt agregando un subdirectorio.

```
add_subdirectory(Escenas)

target_sources(Project_AK PRIVATE Juego.cpp)
```

Ahora, dentro de la carpeta Escenas/, crearemos un CMakeLists.txt para conectar al proyecto todas las escenas que pueden haber.



Contendrá lo siguiente, (Solo toma de la ruta actual, todos los CPP, si no existe ninguno igual toma los HPP).

```
file(GLOB_RECURSE gl_cpp RELATIVE ${CMAKE_CURRENT_SOURCE_DIR} "${CMAKE_CURRENT_SOURCE_DIR}/*.cpp")

target_sources(Project_AK PRIVATE ${gl_cpp})
```

Ahora si, tenemos todo lo que necesitamos para iniciar una escena, crearemos un archivo CPP y un HPP para iniciar la escena.



Iniciaremos por el HPP:

```

/// | ----- |
#pragma once
/// | ----- |
#include "Engine/Layer/Scene.hpp"
#include "Engine/Object/ObjectPool.hpp"
#include "Engine/Render/Batching/RBatch.hpp"
#include "Engine/Render/Batching/RenderEntry.hpp"
#include "Engine/Services/ScenesManager.hpp"
/// | ----- |
#include <vector>
/// | ----- |

namespace APP
{
    class Demo : public ENG::Scene
    {
    public:
        Demo(ENG::SceneID id);

        void Init(void) override;
        void Destroy(void) override;
        bool IsRunning(void) override;
        void Inputs(float dt) override;
        void Update(float dt) override;
        void UpdateFixed(float dt) override;
        void Render(ENG::Batcher& b) override;
    private:
        ENG::ObjectPool pool;
        std::vector<ENG::RenderEntry> renderQueue;
    };
}

```

Si nos damos cuenta, es casi la “misma” secuencia de la GameLayer que previamente creamos, porque si, es relativamente lo mismo, pero aquí ya va toda la lógica del juego para empezar a construirlo.

Ahora continuaremos con el CPP:

```
/// | ----- |
#include "Demo.hpp"
/// | ----- |
#include "Engine/Layer/Scene.hpp"
#include "Engine/Inputs/PollEvent.hpp"
#include "Engine/Services/ScenesManager.hpp"
#include "Engine/Render/Batching/RBatch.hpp"
/// | ----- |

namespace APP
{
    Demo::Demo(ENG::SceneID id)
    : ENG::Scene{}
    {
        sceneID = id;
    }

    void Demo::Init()
    {
        if (isInit)
            return;

        /// Final configurations
        isRunning = true;
        this->renderQueue = pool.Sort();
    }

    void Demo::Inputs(float dt)
    {
        auto& p: PollEvent & = ENG::PollEvent::Get();
    }

    void Demo::Update(float dt)
    {
        for (auto& o: const unsigned int & : pool.GetAllIDs())
        {
            pool.Get(id: o)->Update(dt);
        }
    }

    void Demo::UpdateFixed(float dt)
    {
        (void)dt;
    }
}
```

```

void Demo::Render(ENG::Batcher& b)
{
    for (auto& entry: RenderEntry & : renderQueue)
    {
        pool.Get(id: entry.id)->Draw(& b);
    }
}

void Demo::Destroy()
{
    pool.Clear();
    renderQueue.clear();
    isInit = false;
    isRunning = false;
    tileMaps.clear();
}

bool Demo::IsRunning()
{
    return isRunning;
}
}

```

Listo, tenemos la escena lista en cuanto a código, ahora necesitamos enlazarla al manejador de escenas. Abriremos el Juego.cpp y en OnInit(), agregaremos una escena al manejador de Escenas.

```

#include "Engine/Services/ScenesManager.hpp"
#include "Engine/Services/Services.hpp"
void Game::OnInit(void)
{
    LOG_INFO(message: " | << Game application Init Succesfully");

    // Agregamos la escena Demo
    ENG::Services::Scenes().AddScene(std::make_unique<Demo>(id: ENG::SCENE_DEMO));

    m_currentScene = ENG::Services::Scenes().GetCurrent();
    m_currentScene->Init();
}

```

Aquí tenemos enlazada ya la escena al manejador de escenas, ahora necesitamos crear en la escena principal.

6. ¿Qué es un Objeto?

En Project AK, un objeto es toda aquella entidad que exista en escena, entonces un Botón puede ser un Objeto, el jugador sera un objeto y hasta los textos pueden ser objetos. En que nos beneficia esto, podemos modificar todo a nuestro gusto, desde agregar Shaders, modificar la posición del Objeto cuando queramos, texturas, colores, etc.

Inicialicemos un Objeto, Actualmente por diseño tenemos un ObjectPool, que es la instancia de un manejador de objetos por Escena.

```
void Demo::Init()
{
    if (isInit)
        return;

    /// -> Constructor de Object:  Object(const std::string& name);-
    auto idObjeto: ObjectID = pool.Add(obj: std::make_unique<ENG::Object>(s: "Primer Objeto"));

    /// Final configurations
    isRunning = true;
}
```

Porque todo o mayoría de objetos o entidades que creemos utilizamos:

std :: make_unique <> ()

La razón simple, es que el ownership (Dueño) sera unicamente del pool en este caso, porque nos conviene tener solo un dueño, mantenemos que los objetos solo existan unicamente para el pool actual, y no estén por ahí regados.

Una vez agregado el objeto, este regresara un ObjectID, que es donde se encuentra el objeto en memoria o Dense, para así poder tomarlo desde donde este y modificarlo.

```
void Demo::Init()
{
    if (isInit)
        return;

    /// -> Constructor de Object:  Object(const std::string& name);-
    auto idObjeto: ObjectID = pool.Add(obj: std::make_unique<ENG::Object>(s: "Primer Objeto"));
    /// -> Obtenemos el Objeto sin modificar el ownership.
    auto obj: ENG::Object * = pool.Get(id: idObjeto);
    /// -> Modificamos su posicion actual a {500.0f,500.0f}.
    obj->SetPosition(new_pos: {x: 500.0f,y: 500.0f});

    /// Final configurations
    isRunning = true;
}
```

Listo, en ese momento ya tenemos nuestro primer Objeto creado e inicializado.

7. ¿Qué podemos hacer con un Objeto?

Se pueden hacer muchas cosas, desde agregar sprites (Texturas), animaciones, colores, etc. Lo interesante es que podemos hacer lo que “Queramos” con el objeto.

7.1 Componentes

Los componentes en el Engine son comportamientos que el Objeto puede tomar, pero no necesariamente deba tener si o si. Los únicos componentes que tienen todos los objetos sin excepción son ITransform y IStats.

Existe una lista de los componentes que tiene el Engine:

- IComponents -> Padre de todos los componentes, Clase virtual.
- ITransform -> Controla todo el movimiento del Objeto, dirección y velocidad.
- IStats -> Controla estadísticas del objeto (Vida, Def, etc).
- ISprite -> Controla texturas, atlas y que renderizara el objeto.
- IAnimator -> Controla la lógica de la animación. (Depende de un ISprite obligatoriamente).
- IBoundingBox -> Controla colisiones, es el área del objeto para físicas. (Se actualiza cada frame por ITransform).
- IColor -> Controla el color que pueda almacenar el Objeto.
- IMaterial -> Controla un shader que se le encargue.
- IText -> Almacena text, fuente y el tamaño del texto. (Fuente == Tipo de letra).

Todo esto puede ser agregado como Componente a un Objeto. Como puedo agregar un Componente a un Objeto.

```
// Agregar un Componente a un Objeto.  
// Para inicializar un ISprite:      ISprite(const std::string& key, float scale = 1.0f)  
obj->AddComponent<ENG::ISprite>("Textura1" , 1.0f);
```

Listo, se agrego un ISprite ahora el objeto tendra una textura que previamente se tuvo que cargar.

PARTE II · CÓMO FUNCIONA EL MOTOR

8. Arquitectura general

Antes de escribir código conviene saber quién hace qué. Project AK se apoya en cuatro piezas: SDL3, OpenGL, el propio motor y tu juego. Esta sección explica cómo se conectan y qué ocurre en cada fotograma.

8.1 Las cuatro capas

El proyecto se lee de abajo hacia arriba. Cada capa solo conoce a la de abajo.

Capa	Qué es	De dónde viene
SDL3	Ventana, eventos de entrada, temporizadores y audio. Es la plataforma.	FetchContent (release-3.4.4)
SDL3_mixer	Reproductor de música y efectos de sonido. Se apoya en SDL3.	FetchContent (release-3.2.4)
OpenGL 4.6 Core	Contexto gráfico, shaders y dibujado. El motor habla con la GPU a través de GLAD.	GLAD, generado en el proyecto
Project AK	Ventanas, escenas, objetos, componentes, lote de dibujado, cámara, recursos y audio.	src/Engine/
Tu juego	Tus escenas, tu lógica, tus recursos. El motor no sabe que existe.	src/Game/

Por encima de OpenGL, el motor añade dos bibliotecas de apoyo que debes conocer:

- **GLM (glm)**: matemáticas vectoriales y matriciales (`glm::vec2`, `glm::mat4`). El motor lo usa en el **Batcher** y en **Camera2D**.
- **nlohmann/json**: lectura y escritura de JSON. Se usa para la configuración del motor, las fuentes horneadas y el guardado de partida.

Versión de OpenGL El motor pide un contexto **OpenGL 4.6 Core** y compila los shaders con **#version 460**. La documentación antigua que mencionaba 3.3 está desactualizada: si tu tarjeta no soporta 4.6, actualiza el driver antes de seguir.

8.2 El arranque de la aplicación

Todo empieza en `src/Main/main.cpp`. El punto de entrada se encarga de tres cosas: leer la configuración, construir el motor y por último delegar el juego.

src/Main/main.cpp (esqueleto)

```

int main(void)
{
    ENG::EngineConfig config;
    nlohmann::json fJson;
    auto& path = ENG::Path::Get();

    std::ifstream f(path.ConfigPath / "eng.json");
    if(!f.is_open())
    {
        LOG_ERROR(" | << CONFIGURATION for the engine doesnt exist [IN]: eng.json");
        config.m_eFullscreen = false;
        config.m_title       = "Project_AK";
        config.m_viewWidth   = 1920;
        config.m_viewHeight  = 1080;
    }
    else
    {
        f >> fJson;
        config.Load(fJson);
        f.close();
    }

    auto engine = std::make_unique<ENG::Engine>(config,
std::make_unique<APP::Game>());

    engine->OnInit();          // crea ventana, contexto GL y registra los servicios
    engine->OnRun();            // bucle principal
    engine->OnSave();           // vuelca la configuracion (p.ej. pantalla completa)

    std::ofstream fOut(path.ConfigPath / "eng.json", std::ios::out |
std::ios::trunc);
    fJson = config.Save();
    fOut << fJson;
    fOut.close();

    engine->OnDestroy();
    engine.reset();
    return 0;
}

```

Lee esto **main.cpp** también es el lugar donde se implementa STB (**stb_image**, **stb_image_write**, **stb_truetype**). Se compila en **un** punto del programa, por eso lleva **#define STB_IMAGE_IMPLEMENTATION** justo aquí.

8.3 El bucle principal, fotograma a fotograma

Engine::OnRun() es el corazón del programa. Es un **while** que se repite mientras el juego decida seguir vivo, y su orden no es arbitrario: cada paso depende del anterior.

src/Engine/Engine.cpp (bucle principal, sin comentarios)

```

while (m_game->IsRunning())
{
    clock.Tick();
    auto dt = clock.GetDT();
    PollEvent::Get().ProcessPollEvents();

    m_game->OnUpdateFixed(dt);

    m_game->OnInputs(dt);
    m_game->OnUpdate(dt);
    auto& r = Render::Get();

    r.UpdateRender();

    r.StartDraw();
    {
        ENG::Render::Get().RenderColor(ENG::Color::Gray);
        m_game->OnRender(dt);
    }
    r.EndDraw();

    r.ClearRender();

    PollEvent::Get().ResetPollEvent();
}

```

Orden	Paso	Qué deberías poner ahí
1	clock.Tick() y clock.GetDT()	Nada. Calcula el tiempo del fotograma.
2	PollEvent::ProcessPollEvents()	Nada. Recoge los eventos de SDL de este fotograma.
3	OnUpdateFixed(dt)	Física: movimiento, colisiones, pasos constantes.
4	OnInputs(dt)	Leer teclado, ratón y mando. Sin cambiar el estado del juego.
5	OnUpdate(dt)	Lógica del juego: mover, animar, decidir.
6	r.UpdateRender()	Nada. Refresca el tamaño real de la ventana.
7	r.StartDraw() + RenderColor(...)	Nada. Limpia la pantalla con el color indicado.
8	OnRender(dt)	Enviar sprites, tiles, texto y GUI al lote de dibujado.
9	r.EndDraw()	Nada. Vacía el lote y ejecuta las llamadas de OpenGL.
10	r.ClearRender()	Nada. Intercambia los búferes y muestra el fotograma.
11	PollEvent::ResetPollEvent()	Nada. Limpia los eventos para el siguiente fotograma.

Un error clásico `IsKeyPress` y `IsKeyDown` solo son verdad durante el fotograma en el que se recogió el evento. Si consultas una tecla en `OnRender()` en lugar de `OnInputs()`, el resultado será `false` casi siempre.

9. Servicios: el acceso global

El motor tiene nueve Piece managers. No se pasan como parámetros: se registran una vez y se consultan desde cualquier sitio con `ENG::Services::.....`

`ENG::Services` es un **localizador de servicios**: guarda punteros a los managers y ofrece un acceso estático a cada uno. Quien lo escribe no es el juego, es el propio motor, dentro de `Engine::OnInit()`.

Registro (lo hace el motor, no tú)

```
// src/Engine/Engine.cpp · Engine::OnInit()
Services::ProvideAssets(&m_eAssets);
Services::ProvideScenes(&m_eScenes);
Services::ProvideFonts(&m_eFont);
Services::ProvideSFX(&m_eSFX);
Services::ProvideMusic(&m_eMusic);
Services::ProvideShaders(&m_eShaders);
Services::ProvideWorldSaver(&m_eWorldSaver);
Services::ProvideClockSaver(&m_eClock);
Services::ProvideCSVManager(&m_eCSV);
```

Consulta (esto sí lo haces tú)

```
ENG::Services::Assets().LoadAtlas("Atlas/Isometrico/CaveMap.json", "I_CaveMap");
ENG::Services::Scenes().PeddingScene("MainMenu");
ENG::Services::SFX().PlaySFX("GunShot");
```

Acceso	Devuelve	Para qué sirve
<code>Services::Assets()</code>	<code>AssetsManager&</code>	Cargar texturas, atlas y consultar el array de texturas.
<code>Services::Shaders()</code>	<code>ShaderManager&</code>	Cargar, obtener y asignar materiales (shaders).
<code>Services::Fonts()</code>	<code>FontManager&</code>	Registrar y obtener fuentes horneadas.
<code>Services::Scenes()</code>	<code>ScenesManager&</code>	Registrar escenas y cambiar entre ellas.
<code>Services::Music()</code>	<code>MusicManager&</code>	Cargar, reproducir y cambiar el volumen de la música.
<code>Services::SFX()</code>	<code>SFXManager&</code>	Cargar y disparar efectos de sonido.
<code>Services::WSaver()</code>	<code>WorldSaver&</code>	Guardar y recuperar el estado del mundo.
<code>Services::Clock()</code>	<code>ClockManager&</code>	Consultar <code>dt</code> y el tiempo transcurrido.
<code>Services::CSV()</code>	<code>CSVManager&</code>	Leer tablas de diálogos y traducciones.

Inicialización automática Las funciones **Provide*** de **Assets**, **SFX**, **Music** y **Clock** llaman además a **Init()** sobre el manager recibido. Las demás solo guardan el puntero.

Sin comprobaciones Los accesores desreferencian el puntero sin mirarlo. Si llamas a **Services::Assets()** antes de que el motor haya registrado el manager, el comportamiento es indefinido: revisa que tu código solo se ejecute dentro del bucle.

10. Escenas y el gestor de escenas

Una escena es una pantalla completa con su propio contenido y su propio estado. El juego enseña una escena cada vez, y **ScenesManager** decide cuál.

10.1 Qué es una escena

ENG::Scene es la clase base. Ya trae lo que todas las escenas necesitan: un identificador, un **ObjectPool**, los **TileMap** y una cámara. Lo único que debes implementar es **Render()**.

src/Engine/Layer/Scene.hpp (miembros protegidos que heredas)

```
class Scene
{
public:
    virtual ~Scene() = default;
    virtual void Init(void){};
    virtual void Destroy(void){};
    virtual bool IsRunning(void) {return isRunning;}
    virtual void Inputs(float dt) {(void)dt;};
    virtual void Update(float dt) {(void)dt;};
    virtual void UpdateFixed(float dt) {(void)dt;};
    virtual void Render(ENG::Batcher& b) = 0; // <- la única obligatoria
    std::string GetSceneID(void) const {return this->sceneID;}

protected:
    void AddTileMap(TileMap map);
    void RenderTileMaps(Batcher& b, uint8_t minLayer, uint8_t maxLayer) const;

    bool isRunning = false;
    bool isInit = false;
    std::string sceneID;
    std::unique_ptr<Camera2D> camera;
    std::vector<TileMap> tileMaps;
};
```

Método	Cuándo lo llamas	Para qué
Init()	Al entrar en la escena.	Crear objetos, cargar recursos, colocar la cámara.
Inputs(dt)	Cada fotograma, fase de entrada.	Leer teclado, ratón y mando.
Update(dt)	Cada fotograma, fase de lógica.	Mover objetos, avanzar animaciones.

Método	Cuándo lo llamas	Para qué
UpdateFixed(dt)	Cada fotograma, antes de la entrada.	Física y pasos constantes.
Render(b)	Cada fotograma, fase de dibujado.	Enviar tiles, objetos y GUI al Batcher .
IsRunning()	Cada fotograma.	Devuelve false para pedir el cambio de escena.
Destroy()	Al salir de la escena.	Liberar todo lo que creaste en Init() .

10.2 Registrar y cambiar de escena

Las escenas se registran con un identificador de texto en **Game::OnInit()**. La **primera** que se añade es la que arranca el juego.

src/Game/Game.cpp

```
void Game::OnInit(void)
{
    // shaders primero: las escenas los usan al construirse
    ENG::Services::Shaders().Load("mono.fs", "mono.vs", "Mono");

    ENG::Services::Scenes().AddScene(std::make_unique<MainMenu>("MainMenu"));
    ENG::Services::Scenes().AddScene(std::make_unique<DemoScene>("DemoScene"));

    // la primera escena añadida es la actual
    ENG::Services::Scenes().GetCurrent()->Init();
}
```

Para salir de una escena hay dos pasos: marcar el destino y terminar el fotograma. Este pequeño rodeo existe porque no se puede destruir la escena que está ejecutándose mientras se ejecuta.

src/Game/Scenes/Demo.cpp

```
// 1. la escena se niega a seguir viva y avisa del destino
void DemoScene::Inputs(float dt)
{
    auto& pollEvent = ENG::PollEvent::Get();

    if (pollEvent.IsKeyPress(SDL_SCANCODE_P))
    {
        isRunning = false;
        ENG::Services::Scenes().PeddingScene("MainMenu");
        ENG::Services::Music().StopMusic("Marine");
    }
}

// 2. Game::OnUpdate() ve isRunning == false y hace el cambio
void Game::OnUpdate(float dt)
{
    auto s_current = ENG::Services::Scenes().GetCurrent();
    if(!s_current->IsRunning())
    {
        s_current->Destroy();
        ENG::Services::Scenes().ChangeScene();
        ENG::Services::Scenes().GetCurrent()->Init();
        return;
    }
    s_current->Update(dt);
}
```

Función	Qué hace
AddScene(std::unique_ptr<Scene>)	Registra una escena. La primera se marca como actual.
GetCurrent()	Devuelve un puntero a la escena activa.
PeddingScene("id")	Marca el destino. No cambia nada todavía.
ChangeScene()	Ejecuta el cambio pendiente y pone la cámara del Batcher a nullptr .
DestroyScene("id")	Elimina una escena del registro.
Clear()	Vacía el registro. Lo llama Engine::OnDestroy() .

10.3 TileMaps dentro de una escena

Una escena puede tener varios mapas de tiles, cada uno en su propia capa. Se añaden al construirse y se dibujan por rangos.

Cargar un mapa exportado de Tiled

```
// el atlas que usará el mapa
ENG::Services::Assets().LoadAtlas("Atlas/Isometrico/CaveMap.json", "I_CaveMap");

// el mapa: un TileMap por cada capa del JSON de Tiled
for (auto& mapLayer : ENG::TileMap::LoadTiledMap("I_CaveMap",
"Map/Scenes/expIsometrico.json"))
{
    AddTileMap(std::move(mapLayer));
}
```

Dibujarlos por capas en Render()

```
void DemoScene::Render(ENG::Batcher& b)
{
    RenderTileMaps(b, LAYER_GROUND, LAYER_PLAYER); // suelo: detrás del jugador
    for (auto& entry : renderQueue)
    {
        auto* obj = pool.Get(entry.id);
        obj->Draw(b);
    }
    RenderTileMaps(b, LAYER_PLAYER, LAYER_MAX); // vegetation y decoracion:
    delante
}
```

Por qué dos llamadas Dibujar el mapa en dos tramos es lo que permite que el jugador pase **por detrás** de los árboles: las capas altas del mapa se dibujan después que el jugador.

11. Objetos y el ObjectPool

El **ObjectPool** es el dueño de todos los objetos de una escena. Aporta identificadores estables, búsqueda en $O(1)$ y el orden de dibujado.

11.1 Añadir, buscar y quitar

El pool se declara como miembro de la escena, no como variable local: debe vivir mientras la escena viva.

Ciclo básico de un objeto

```
// src/Game/Scenes/Demo.cpp
auto idPlayer = pool.Add(std::make_unique<ENG::Object>("Player")); // devuelve
ObjectID
auto objPlayer = pool.Get(idPlayer); // devuelve
Object*
objPlayer->SetPosition(ENG::Vector2{500, 500});
objPlayer->AddComponent<ENG::IBoundingBox>(ENG::Vector2{16, 16});

// ...en cualquier momento posterior...
ENG::Object* p = pool.Get(idPlayer);
if (p) { p->Update(dt); }

pool.Remove(idPlayer); // libera el objeto
pool.Clear(); // vacía el pool entero (típico en Destroy())
```

Función	Devuelve	Notas
<code>Add(std::unique_ptr<Object>)</code>	<code>ObjectID</code>	Los identificadores empiezan en 1.
<code>Get(id)</code>	<code>Object*</code> o <code>nullptr</code>	Búsqueda en O(1).
<code>Remove(id)</code>	—	Intercambia con el último y hace pop_back .
<code>GetAllIDs()</code>	<code>const std::vector<ObjectID>&</code>	Para recorrer la escena entera.
<code>Sort()</code>	<code>const std::vector<RenderEntry>&</code>	Ordena por capa y shader.
<code>Size()</code>	<code>int</code>	Cuántos objetos hay.
<code>Clear()</code>	—	Vacía el pool y reinicia el contador de identificadores.

11.2 Cómo se ordenan los objetos al dibujar

`Sort()` construye la cola de dibujo agrupando por número de capa y, dentro de la misma capa, por shader. La ordenación es **estable**, así que dos objetos en la misma capa conservan el orden en el que se añadieron.

src/Engine/Object/ObjectPool.hpp (extracto de `Sort()`)

```
const std::vector<RenderEntry>& Sort()
{
    if(!sortedQueue)
    {
        renderQueue.clear();
        for (ENG::ObjectID id : denseID)
        {
            auto obj = Get(id);
            renderQueue.push_back({ id, obj->GetLayer(), obj->GetShader() });
        }

        std::stable_sort(renderQueue.begin(), renderQueue.end(),
            [](const ENG::RenderEntry& a, const ENG::RenderEntry& b)
            {
                if(a.layer != b.layer) return a.layer < b.layer;
                return a.shader < b.shader;
            });
        sortedQueue = true;
    }
    return renderQueue;
}
```

Solo se ordena una vez La bandera `sortedQueue` evita reordenar en cada fotograma. Tanto `Add()` como `Remove()` la ponen a `false`, así que la siguiente llamada a `Sort()` rehace el trabajo.

11.3 Capas de dibujado

El número de capa es un `uint8_t` y define el orden de superposición. El motor trae estos valores ya definidos; úsalos en lugar de inventar números.

`src/Engine/Map/TileMap/TileMap.hpp`

```
#define LAYER_GROUND      0
#define LAYER_WORLD      10
#define LAYER_PLAYER      20
#define LAYER_MIDDLE      25
#define LAYER_TOP         30
#define LAYER_FX          40
#define LAYER_UI          50
#define LAYER_MAX         255
```

Capa	Valor	Uso típico
<code>LAYER_GROUND</code>	0	Suelo, fondo del nivel.
<code>LAYER_WORLD</code>	10	Decoración del mundo, detrás del jugador.
<code>LAYER_PLAYER</code>	20	El jugador y los entidades.
<code>LAYER_MIDDLE</code>	25	Elementos intermedios.
<code>LAYER_TOP</code>	30	Copas de árboles, techos, follaje.
<code>LAYER_FX</code>	40	Partículas, explosiones, destellos.
<code>LAYER_UI</code>	50	Interfaz de usuario.
<code>LAYER_MAX</code>	255	Último valor válido; útil como límite superior.

12. Componentes en profundidad

La sección 7.1 presenta la lista de componentes del motor. Aquí entramos en el cómo: qué hay por dentro, cómo se añaden y cómo se consultan.

12.1 Cómo funcionan por dentro

Todos los componentes heredan de `IComponents` y viven en un `ComponentStorage` que pertenece al objeto. El almacenamiento es un mapa indexado por `std::type_index`, de modo que **solo puede haber un componente de cada tipo**.

src/Engine/Component/ComponentStorage.hpp

```

template<typename T, typename... Args>
T& Eplace(Args&&... args)
{
    auto key = std::type_index(typeid(T));
    auto ptr = std::make_unique<T>(std::forward<Args>(args)...);
    _map[key] = std::move(ptr);
    return *_map[key];
}

template<typename T>
T* Get()
{
    auto it = _map.find(std::type_index(typeid(T)));
    if(it == _map.end()) return nullptr;
    return static_cast<T*>(it->second.get());
}

template<typename T>
bool Has()
{
    return _map.contains(std::type_index(typeid(T)));
}

```

Un componente por tipo Como la clave es el tipo, llamar dos veces a **AddComponent<ISprite>()** **reemplaza** el sprite anterior en lugar de añadir un segundo. Para tener varios componentes del mismo tipo necesitas una clase propia.

12.2 Añadir, consultar y comprobar

src/Engine/Object/Object.hpp (API pública)

```

template<typename T, typename... Args>
T& AddComponent(Args&&... args) { return
components.Eplace<T>(std::forward<Args>(args)...); }

template<typename T> T* GetComponent() const { return components.Get<T>(); }
template<typename T> bool HasComponent() const { return components.Has<T>(); }

```

Uso típico

```

auto& sprite = objPlayer->AddComponent<ENG::ISprite>("Player.png");
objPlayer->AddComponent<ENG::IColor>(ENG::Color::Red);
objPlayer->AddComponent<ENG::IMaterial>("Mono");

if (auto* box = objPlayer->GetComponent<ENG::IBoundingBox>())
{
    box->ChangeSize(ENG::Vector2{32, 32});
}

if (objPlayer->HasComponent<ENG::IAnimator>())
{
    objPlayer->GetComponent<ENG::IAnimator>()->Play();
}

```

12.3 Catálogo de componentes

Componente	Responsabilidad	Notas
IComponents	Clase base virtual de todos los componentes.	No se usa directamente.
ITransform	Posición, dirección y velocidad.	Todos los objetos lo tienen.
IStats	Estadísticas: vida, defensa, etc.	Todos los objetos lo tienen.
ISprite	Textura, atlas o región que se dibuja.	Admite tres constructores distintos.
IAnimator	Avance de la animación por fotogramas.	Necesita un ISprite .
IBoundingBox	Caja de colisión del objeto.	Se actualiza cada fotograma con el transform.
IColor	Tinte RGBA del objeto.	Afecta al dibujado.
IMaterial	Shader asignado al objeto.	El puntero se usa para agrupar en el Sort() .
IText	Cadena de texto y tamaño.	No confundir con la clase ENG::Text .
IDialog	Diálogo CSV con efecto de escritura.	Usa Services::CSV() .

12.4 Los tres constructores de ISprite

src/Engine/Component/Component.hpp

```
ISprite(const std::string& key, float scale = 1.0f);
// textura suelta
ISprite(const std::string& atlasKey, const std::string& animName,
        int frameIndex, float scale = 1.0f);
// fotograma de animación
ISprite(const std::string& atlasKey, const std::string& tileID,
        float scale = 1.0f);
// tile de un atlas
```

Animaciones

```
// por número de fotogramas (el atlas se recorre en horizontal)
obj->AddComponent<ENG::IAnimator>("CatAtlas", 4, 8.0f); // 4 frames a 8 fps

// por regiones explícitas del atlas
std::vector<ENG::Rectangle> rects = { {0, 0, 64, 64}, {64, 0, 64, 64} };
obj->AddComponent<ENG::IAnimator>("CatAtlas", rects, 8.0f);

auto* anim = obj->GetComponent<ENG::IAnimator>();
anim->Play();
anim->Pause();
anim->Resume();
anim->Stop();
anim->Advance(dt); // también se llama desde Object::Update()
```

Orden de los argumentos El tercer parámetro de **IAnimator** es la **velocidad en fotogramas por segundo**, no el número de fotogramas. El número de fotogramas va antes, y **step** (cada cuántos fotogramas avanza) es el último argumento, con valor por defecto 1.

13. Entrada: teclado, ratón y mando

Toda la entrada pasa por una única clase, **ENG::PollEvent**. El motor la vacía al final de cada fotograma, así que solo hay que preguntar durante **Inputs()**.

13.1 El ciclo de los eventos

- **PollEvent::Get()** devuelve la única instancia (singleton) del motor.
- **ProcessPollEvents()** (lo llama **Engine::OnRun()**) recorre los eventos de SDL de este fotograma.
- Tus consultas (**IsKeyPress**, **IsKeyDown**, ...) leen ese estado.
- **ResetPollEvent()** (también automático) borra el estado para el siguiente fotograma.

src/Game/Game.cpp

```
void Game::OnInputs(float dt)
{
    auto& pollEvent = ENG::PollEvent::Get();

    if(pollEvent.IsKeyPress(SDL_SCANCODE_ESCAPE))
        m_gameRunning = !m_gameRunning;

    if(pollEvent.IsKeyPress(SDL_SCANCODE_F1))
        ENG::Render::Get().ToggleFullscreen();

    // y el resto de la entrada se delega en la escena activa
    ENG::Services::Scenes().GetCurrent()->Inputs(dt);
}
```

13.2 Referencia de la API

Pregunta	Devuelve	Nota
IsKeyPress(SDL_Scancode)	bool	Solo el fotograma en el que se pulsó.
IsKeyDown(SDL_Scancode)	bool	Mientras se mantenga pulsada.
IsKeyRelease(SDL_Scancode)	bool	Solo el fotograma en el que se soltó.
IsMouseButtonPress(SDL_MouseButton)	bool	Clic izquierdo, derecho o central.
IsMouseButtonDown(SDL_MouseButton)	bool	Botón mantenido.
IsMouseButtonRepeat(button, times)	bool	Pulsaciones repetidas por el sistema.

Pregunta	Devuelve	Nota
<code>IsMouseButtonRelease(SDL_MouseButton)</code>	<code>bool</code>	Botón soltado.
<code>GetMouseWheel()</code>	<code>float</code>	Desplazamiento vertical de la rueda.
<code>GetMousePosition()</code>	<code>ENG::Vector2</code>	Está en <code>ENG::Mouse</code> , no en <code>PollEvent</code> .

El mando todavía no está implementado Las cuatro funciones de gamepad (`IsGamepadButtonPress`, `IsGamepadButtonDown`, `IsGamepadAxisMove`) están declaradas en la cabecera pero su implementación devuelve siempre `false`. No las uses todavía: no Reading nada.

13.3 Distinguir pulsación de mantenimiento

Esta es la confusión más común al empezar. `IsKeyPress` detecta el flanco de bajada (el instante en el que se pulsa); `IsKeyDown` devuelve `true` en todos los fotogramas que la tecla esté abajo.

Movimiento continuo frente a acción puntual

```
// CONTINUO: mientras W esté abajo (src/Game/Scenes/Demo.cpp)
if (pollEvent.IsKeyDown(SDL_SCANCODE_W))
    pool.Get(PLAYER)->GetTransform().m_direction.y -= 1.0f;

// PUNTUAL: solo el fotograma de la pulsacion
if (pollEvent.IsKeyPress(SDL_SCANCODE_P))
{
    isRunning = false;
    ENG::Services::Scenes().PeddingScene("MainMenu");
}
```

Traduce teclas a ejes Un `float` que se suma o resta cada fotograma acaba growing sin límite. La forma limpia es mantener una dirección y normalizarla, como hace `System_PlayerMovement()`.

14. El renderizador y el lote de dibujado

Dibujar objeto por objeto con una llamada de OpenGL cada uno sería lentísimo. El motor no lo hace: acumula todo el fotograma y lo envía de una vez.

14.1 El Batcher

`ENG::Batcher` (alias `RAPIBatch`) es un lote de dibujado. Durante `OnRender()` accumulates vértices, índices y ranuras de textura; en `EndDraw()` los vuelca todos juntos.

Constante	Valor	Significado
<code>MAX_QUADS</code>	10 000	Cuadrados máximos por fotograma.

Constante	Valor	Significado
MAX_VERTS	40 000	4 vértices por cuadrado.
MAX_INDICES	60 000	6 índices por cuadrado.
MAX_TEXTURES	16	Texturas simultáneas en el lote.
ARRAY_TEXTURE_UNIT	16	Unidad de textura donde vive el array de texturas.

La ranura 0 está ocupada De las 16 ranuras de textura, la número 0 la usa una textura blanca fija. En la práctica te quedan **15** texturas distintas por fotograma. Si te pasas, el motor reutiliza la última ranura y verás la imagen equivocada.

14.2 Primitivas de dibujo

src/Engine/Render/Batching/RBatch.hpp (API del lote)

```
void SetCamera2D(ENG::Camera2D* camera);
void SetMaterial(Shader* _Material);

void DrawTexture (const glm::vec2& pos, const glm::vec2& size,
std::shared_ptr<RImage> texture,
                const glm::vec4& tint = {1,1,1,1}, float effectMode = 0.0f);
void DrawAtlasSprite(const glm::vec2& pos, const glm::vec2& size, int layer,
                const glm::vec2& uvMin, const glm::vec2& uvMax,
                const glm::vec4& tint = {1,1,1,1}, float effectMode = 0.0f);
void DrawGlyph (const glm::vec2& pos, const glm::vec2& size,
std::shared_ptr<RImage> fontAtlas,
                const glm::vec2& uvMin, const glm::vec2& uvMax,
                const glm::vec4& tint = {1,1,1,1}, float effectMode = 0.0f);

void DrawQuad (const glm::vec2& pos, const glm::vec2& size, const
glm::vec4& color);
void DrawQuadOutline (float x, float y, float w, float h, const Color& color,
float thicknes = 1.0f);
void DrawCircle (const glm::vec2& center, float radius, const glm::vec4&
color, uint32_t segments = 32);
void DrawCircleOutLine(const glm::vec2& center, float radius, const glm::vec4&
outcolor, float thickness = 1.0f);
void DrawTriangle (const glm::vec2& a, const glm::vec2& b, const glm::vec2&
c, const glm::vec4& color);
void DrawLine (const glm::vec2& a, const glm::vec2& b, float thickness,
const glm::vec4& color);
void DrawPolygon (const std::vector<glm::vec2>& points, const glm::vec4&
color);
```

Para lo decoration del día a día existe además **ENG::Drawer**, una fachada con las mismas primitivas pero firmas más cortas y colores **ENG::Color** ya definidos.

src/Game/Game.cpp y Demo.cpp

```

auto& b      = ENG::Render::Get().GetBatcher();
auto screen = ENG::Render::Get().GetScreenSize();

// líneas de ayuda para posicionar cosas
ENG::Drawer::DrawLine({screen.x / 2, 0.0f}, {screen.x / 2, screen.y},
ENG::Color::Red);
ENG::Drawer::DrawLine({0.0f, screen.y / 2.0f}, {screen.x, screen.y / 2.0f},
ENG::Color::Red);

// un rectángulo de prueba
ENG::Drawer::DrawRectangle({0, 0, 100, 100}, ENG::Color::Blue);

```

14.3 El objeto Render

Función	Qué hace
Render::Get()	Devuelve el singleton.
InitWindowSDLContext(title, w, h)	Crea la ventana. Devuelve 0 si falla.
InitGLContext()	Crea el contexto OpenGL.
InitRenderContext(EngineConfig&)	Compila los shaders base y ajusta el viewport.
SetScreenSize(w, h)	Fija el tamaño de pantalla de la cámara.
GetScreenSize()	Devuelve ENG::Vector2 con el tamaño actual.
GetFullscreen() / SetFullscreen(b)	Consulta y cambia pantalla completa.
ToggleFullscreen()	Alterna entre ventana y pantalla completa.
GetBatcher()	Devuelve el Batcher& que usará OnRender() .
RenderColor(Color)	Color de limpieza de la pantalla.
StartDraw() / EndDraw()	Inicio y fin del lote.
ClearRender()	Intercambia los búferes.
Destroy()	Libera ventana, contexto y lote.

14.4 El ciclo de dibujado de un objeto

Un objeto no dibuja: **entrega datos**. **Object::Draw(Batcher&)** llama a las funciones del lote, y el lote se encarga del resto. Si sobrescribes **Draw()**, llama a **b** y no hagas nada más.

Sobrescritura típica

```
class Enemigo : public ENG::Object
{
public:
    Enemigo(const std::string& nombre) : ENG::Object(nombre) {}

    void Draw(ENG::Batcher& b) const override
    {
        auto* sprite = GetComponent<ENG::ISprite>();
        if (!sprite) return;

        b.SetMaterial(GetShader());
        b.DrawTexture(GetPosition(), GetSize(), sprite->GetImage());
    }

    void Update(float dt) override
    {
        ENG::Object::Update(dt);    // llama a los componentes
        // aqui tu logica
    }
};
```

Por qué se ordenan por shader Dentro de una misma capa, **ObjectPool::Sort()** agrupa por puntero de shader. Es una optimización clásica: cambiar de shader obliga a OpenGL a reconfigurar el estado, y agrupar minimiza esos cambios.

15. La cámara 2D

La cámara decide qué parte del mundo se ve. **Camera2D** produce la matriz vista-proyección que el **Batcher** aplica a todo lo que se dibuja.

15.1 Crearla y conectarla

src/Game/Scenes/Demo.cpp

```
auto r = ENG::Render::Get().GetScreenSize();
cam = std::make_unique<ENG::Camera2D>(r.x, r.y);
cam->SetTarget(&objPlayer->GetTransform()); // sigue al jugador
cam->SetPosition(objPlayer->GetPosition());
cam->SetZoom(2.0f);

ENG::Render::Get().GetBatcher().SetCamera2D(cam.get());
```

No olvides registrarla Crear la cámara no basta: hay que pasarla al **Batcher** con **SetCamera2D()**. Además, **ScenesManager::ChangeScene()** pone el puntero del **Batcher** a **nullptr**, así que **cada escena debe conectar su propia cámara** en su **Init()**.

15.2 API de Camera2D

Función	Efecto
Camera2D(w, h)	Crea la cámara con el tamaño del viewport.
SetPosition(x, y) / SetPosition(Vector2)	Mueve la cámara.
Move(Vector2 delta)	Desplaza la cámara.
SetTarget(const ITransform*)	La cámara sigue a un objeto.
SetTarget(const IBoundingBox*)	La cámara sigue el centro de una caja.
SetZoom(float) / GetZoom()	Zoom. Valores altos se acercan.
SetRotation(float rad) / GetRotation()	Rotación en radianes.
Resize(w, h) / Resize(Vector2)	Actualiza el tamaño del viewport.
GetViewportSize()	Tamaño actual del viewport.
GetViewProjectionMatrix()	Matriz final que usa el lote.
ScreenToWorld(Vector2)	Convierte coordenadas de pantalla a mundo.

15.3 Seguimiento suave con la rueda

Zoom con la rueda del ratón

```
float wheel = pollEvent.GetMouseWheel();
if (wheel != 0.0f)
{
    float zoom = cam->GetZoom() + wheel * 0.1f;
    cam->SetZoom(std::clamp(zoom, 1.0f, 4.0f));
}
```

Ojo con el signo **GetMouseWheel()** devuelve positivo al girar hacia arriba. Si el zoom va al revés de lo que esperas, invierte el signo antes de aplicarlo.

16. Recursos: texturas, atlas y el array de texturas

El gestor de recursos evita cargar el mismo archivo dos veces y Agrupa los gráficos en un array de texturas de OpenGL para poder mezclarlos en un solo lote.

16.1 Cargando recursos

Todo lo que se carga se identifica con una **clave** de texto. Después se recupera con esa misma clave; la ruta del archivo solo se usa al cargar.

```
src/Game/Scenes/Demo.cpp
```

```
// textura suelta
ENG::Services::Assets().Load("Entities/Entites.png", "Entities");

// atlas (JSON + PNG): devuelve false si algo falla
bool ok = ENG::Services::Assets().LoadAtlas("Atlas/Isometrico/CaveMap.json",
"I_CaveMap");

// recuperar
std::shared_ptr<ENG::RImage> img =
ENG::Services::Assets().GetTexture("Entities");
const ENG::AtlasData* atlas      =
ENG::Services::Assets().GetAtlas("I_CaveMap");
```

Función	Devuelve	Para qué
<code>Load(path, keyName)</code>	—	Carga una imagen suelta y la registra con su clave.
<code>LoadAtlas(path, key)</code>	<code>bool</code>	Carga un atlas desde su JSON.
<code>GetTexture(keyName)</code>	<code>std::shared_ptr<RImage></code>	Devuelve la textura ya cargada.
<code>GetAtlas(key)</code>	<code>const AtlasData*</code>	Datos del atlas: regiones, tiles, tamaño.
<code>GetAtlasByTexture(path)</code>	<code>const AtlasData*</code>	Busca el atlas a partir del PNG.
<code>GetTextureArrayID()</code>	<code>GLuint</code>	El array de texturas de OpenGL.
<code>Init()</code>	—	La llama <code>ProvideAssets()</code> . Crea el array de 2048×2048×16.
<code>Clear()</code>	—	Libera todo lo cargado.

El array de texturas tiene un tamaño fijo Al arrancar se reserva un array de **2048×2048 píxeles y 16 capas**. Un atlas no puede superar esas dimensiones y el número de atlas está acotado por las 16 capas.

16.2 Qué contiene un atlas

Un atlas se describe con un JSON (**ParseAtlasJSON**) que apunta a un PNG. De ese JSON el motor extrae las regiones de animación y los tiles.

Dato	Tipo	Significado
<code>atlasWidth / atlasHeight</code>	<code>int</code>	Tamaño del PNG del atlas.
<code>tileWidth / tileHeight</code>	<code>int</code>	Tamaño de una celda del atlas.

Dato	Tipo	Significado
<code>atlasLayer</code>	<code>int</code>	Capa del array de texturas donde vive.
<code>containerWidth / containerHeight</code>	<code>int</code>	Tamaño del PNG que se exportó.
<code>animations</code>	<code>map<string, AnimationRegion></code>	Regiones de animación por nombre.
<code>tiles</code>	<code>map<string, TileRegion></code>	Tiles por identificador de texto.
<code>tilesById</code>	<code>map<int, TileRegion></code>	Tiles por identificador numérico.

16.3 Coordenadas de textura (UV)

`src/Engine/Render/Image/AtlasData.hpp`

```
struct UVRect
{
    glm::vec2 uvMin;
    glm::vec2 uvMax;
    glm::vec2 pixelSize;
};

// helpers
UVRect GetFrameUV(const AtlasData& atlas, const std::string& animName, int
frameIndex);
UVRect GetTileUV (const AtlasData& atlas, const std::string& tileID);
UVRect GetTileUV (const AtlasData& atlas, int tileID);
```

Estructura recomendada Guarda en el proyecto una hoja de sprites por personaje o por escenario y un JSON que la describa. Así el motor carga una sola imagen y tú accedes a cada región por nombre, en vez de cargar decenas de archivos.

17. Shaders y materiales

Un shader es el programa que se ejecuta en la GPU. El motor lo carga una vez y lo reparte como "material" a los objetos que lo necesitan.

17.1 Cargar un shader

`src/Game/Game.cpp`

```
ENG::Services::Shaders().Load("mono.fs", "mono.vs", "Mono");
```

Los archivos se leen de la carpeta `shaders/` junto al ejecutable, no de `assets/`.

Archivos incluidos en el proyecto

```

shaders/
├── mono.vs      · vértice monocromático (material por defecto de los
objetos)
├── mono.fs
└── gl/
    ├── texture.vs  · vértice de textura
    └── texture.fs

```

17.2 Asignar un material a un objeto**src/Engine/Services/ShaderManager.hpp y Component.hpp**

```

Shader* Get(const std::string& idKey);      // busca el shader
void Draw(const std::string& idKey);        // dibujado inmediato con ese
shader

// asignarlo a un objeto
obj->AddComponent<ENG::IMaterial>("Mono");

// o cambiar el material activo del lote
b.SetMaterial(ENG::Services::Shaders().Get("Mono"));

```

El material va en el orden de dibujado Como **ObjectPool::Sort()** agrupa por shader, todos los objetos con el mismo material salen juntos. Si cambias el material de un objeto después de ordenar la cola, llama otra vez a **Sort()**.

17.3 Anatomía de un shader

El motor exige GLSL 4.60. Los dos shaders incluidos usan el mismo esquema de vértices, de modo que cualquier material nuevo debe respetar esos **layout(location = ...)**.

shaders/gl/texture.vs (extracto)

```
#version 460 core

layout(location = 0) in vec3  a_Position;
layout(location = 1) in vec4  a_Color;
layout(location = 2) in vec2  a_TexCoord;
layout(location = 3) in float a_TexIndex;      // ranura 0..15 del lote
layout(location = 4) in float a_ArrayLayer;    // capa del array de atlas
layout(location = 5) in float a_EffectMode;    // 0 ninguno, 1 mono, 2 flash, 3 veneno

out vec4  v_Color;
out vec2  v_TexCoord;
flat out int  v_TexIndex;
flat out float v_ArrayLayer;
flat out float v_EffectMode;

void main()
{
    v_Color      = a_Color;
    v_TexCoord   = a_TexCoord;
    v_TexIndex   = int(a_TexIndex);
    v_ArrayLayer = a_ArrayLayer;
    v_EffectMode = a_EffectMode;
    gl_Position = u_ViewProjection * vec4(a_Position, 1.0);
}
```

shaders/gl/texture.fs (extracto)

```
layout(binding = 0)  uniform sampler2D      u_Textures[16];
layout(binding = 16) uniform sampler2DArray u_AtlasArray;

out vec4 FragColor;

void main()
{
    vec4 texColor;
    if (v_ArrayLayer >= 0.0) {
        texColor = texture(u_AtlasArray, vec3(v_TexCoord, v_ArrayLayer));
    } else {
        switch(v_TexIndex) {          // 16 casos, uno por ranura
            case 0: texColor = texture(u_Textures[0],  v_TexCoord); break;
            // ...
            case 15: texColor = texture(u_Textures[15], v_TexCoord); break;
            default: texColor = vec4(1.0); break;
        }
    }

    FragColor = texColor * v_Color;
}
```

Efecto	effectMode	Qué hace
Normal	0.0f	El tinte se aplica tal cual.
Mono	1.0f	Convierte a escala de grises.

Efecto	<code>effectMode</code>	Qué hace
Flash	<code>2.0f</code>	Pinta de blanco (destello de impacto).
Veneno	<code>3.0f</code>	Tinte verde pulsante usando <code>u_Time</code> .

El efecto va por vértice `effectMode` es un atributo de vértice, no un estado del lote. Pasarlo a `DrawTexture`, `DrawGlyph` o `DrawAtlasSprite` cambia el efecto de ese sprite concreto.

Los dos shaders incluidos `mono.vs/mono.fs` fuerza escala de grises siempre y no acepta efectos; `gl/texture.vs/gl/texture.fs` es el material completo, con UBO por fotograma y efectos. Para trabajo normal, usa el segundo.

18. TileMaps

Un **TileMap** dibuja un mapa de cuadrículas tomando las celdas de un atlas. El motor puede leer directamente el JSON que exporta Tiled.

18.1 Cargar un mapa de Tiled

`LoadTiledMap()` lee el JSON que exporta Tiled y devuelve un **TileMap** por cada capa de tipo `tilelayer`. El atlas se indica aparte y se usa para resolver los identificadores de celda.

`src/Game/Scenes/Demo.cpp`

```
ENG::Services::Assets().LoadAtlas("Atlas/Isometrico/CaveMap.json", "I_CaveMap");

for (auto& mapLayer : ENG::TileMap::LoadTiledMap("I_CaveMap",
"Map/Scenes/expIsometrico.json"))
{
    AddTileMap(std::move(mapLayer));
}
```

El motor decide la capa de dibujado a partir del **nombre** de la capa en Tiled. Si el nombre no está en la tabla, la capa cae en `LAYER_GROUND`.

Nombre de la capa en Tiled	Capa asignada
Ground	<code>LAYER_GROUND</code> (0)
World	<code>LAYER_WOLRD</code> (10)
Middle	<code>LAYER_MIDDLE</code> (25)
Top	<code>LAYER_TOP</code> (30)
FX	<code>LAYER_FX</code> (40)
UI	<code>LAYER_UI</code> (50)
cualquier otro	<code>LAYER_GROUND</code> (0)

Un solo tileset por mapa La resolución de celdas usa un único **firstgid** (el del primer tileset) y calcula **gid - firstgid** como índice dentro del atlas. Si tu mapa usa varios tilesets, los identificadores de los siguientes no se resolverán bien. Siempre incluye un tileset grande con todas las texturas posibles.

18.2 Orientación del mapa

Valor de orientation	Modo	Resultado
"orthogonal"	TileOrientation::Orthogonal	Mapa de cuadrillas clásico (vistas de frente).
"isometric"	TileOrientation::Isometric	Mapa isométrico con rombos.
cualquier otro	Orthogonal	Es el valor por defecto.

18.3 API de TileMap

Función	Devuelve	Notas
LoadTiledMap(atlasKey, mapPath, scale)	std::vector<TileMap>	Un mapa por cada tilelayer .
Render(Batcher& b)	—	Envía las celdas no vacías al lote.
RenderOrthogonal(b, rect)	—	Ruta privada de dibujado ortogonal.
RenderIsometric(b, rect)	—	Ruta privada de dibujado isométrico.
SetPosition(Vector2) / GetPosition()	Vector2	Desplaza el mapa.
SetLayer(uint8_t) / GetLayer()	uint8_t	Fuerza la capa de dibujado.
GetName()	const std::string&	Nombre de la capa en Tiled.
GetWidth() / GetHeight()	int	Tamaño en celdas.
GetLayerNum(name)	uint8_t	Traduce un nombre de capa a número.

La celda 0 no se dibuja En el JSON de Tiled, un **gid** de 0 significa "celda vacía". El motor lo guarda como **-1** y lo omite al dibujar, así que no necesitas limpiar nada a mano.

18.4 Dibujar un subrange de capas

La idea clave del dibujado de mapas es **RenderTileMaps(b, min, max)**, que dibuja solo los TileMap cuyo número de capa esté en el intervalo. Ese intervalo es lo que crea la ilusión de profundidad.

src/Game/Scenes/Demo.cpp · Render()

```
void DemoScene::Render(ENG::Batcher& b)
{
    // 1. todo lo que va detras del jugador
    RenderTileMaps(b, LAYER_GROUND, LAYER_PLAYER);

    // 2. los objetos, en el orden que dicta ObjectPool::Sort()
    for (auto& entry : renderQueue)
    {
        auto* obj = pool.Get(entry.id);
        obj->Draw(b);
    }

    // 3. todo lo que va delante del jugador
    RenderTileMaps(b, LAYER_PLAYER, LAYER_MAX);
}
```

La receta Capa **Ground** detrás, jugador en 20, capa **Top** delante. Ese es el patrón para que el personaje pase por debajo de un árbol o por detrás de un arco.

La posición inicial del mapa Al cargar, **LoadTiledMap()** coloca cada mapa en **{0,0}**. Si tu mapa aparece desplazado, casi siempre es por esto: corrígelo con **SetPosition()** o con la cámara.

19. Matemáticas del juego: Vector2, Color y geometría

El motor trae sus propios tipos de matemáticas para que el código del juego no dependa de una biblioteca externa. Son sencillos, pero tienen algunas sorpresas.

19.1 Vector2

ENG::Vector2 es un par de **float** con álgebra básica y conversión implícita a **glm::vec2**.

src/Engine/Utils/Vector2.hpp (extracto)

```

class Vector2
{
    public:
        Vector2() {}
        explicit Vector2(float _) : x(_), y(_) {}           // un solo valor para
ambos ejes
        Vector2(float x, float y) : x(x), y(y) {}

        void SetVector(float x, float y);
        void Normalize();                                   // divide por la
longitud, si no es cero

        float Length(const Vector2& v);                     // devuelve la longitud

        operator glm::vec2() const;                         // conversion implicita

        // operadores: + - * (Vector2 y float), / (solo float), == != < <= > >=
        // y las versiones de asignación: +=, -=
};

```

Cómo evitar las trampas

```

// MAL: devuelve la longitud del otro vector
float d = a.Length(b);

// BIEN: normaliza el propio vector
a.Normalize();
float d = std::sqrt(a.x * a.x + a.y * a.y);

// division componente a componente
Vector2 SafeDiv(const Vector2& v, float k)
{
    if (k == 0.0f) return {0.0f, 0.0f};
    return {v.x / k, v.y / k};
}

```

Restricción de componentes El comportamiento del operador `/` sugiere que el motor asume que los vectores de posición tienen las dos componentes distintas de cero. Si trabajas con ejes (por ejemplo, solo el eje X), usa **SafeDiv()** o multiplica por el inverso.

19.2 Color

ENG::Color guarda cuatro **float** entre 0 y 1, con conversión implícita a **glm::vec4** y una lista de colores predefinidos.

src/Engine/Render/Color/RColor.hpp

```
class Color
{
public:
    constexpr Color() : r(1), g(1), b(1), a(1) {};
    constexpr Color(float r, float g, float b, float a = 1);

    void From255(uint8_t _r, uint8_t _g, uint8_t _b, uint8_t _a = 255);
    operator glm::vec4() const;

    static const Color Red, Green, Yellow, Blue, White, Black, Gray, Blank;
};
```

Uso

```
ENG::Color rojo    = ENG::Color::Red;
ENG::Color verde   = ENG::Color::Green;
ENG::Color Morado{0.6f, 0.2f, 0.8f, 1.0f};
ENG::Color desde255;
desde255.From255(230, 57, 70);           // 0-255, como en CSS
```

19.3 Geometría

src/Engine/Utils/RawGeometry.hpp

```
struct Rectangle { float x; float y; float w; float h; };
struct Circle    { Vector2 center; float radius; };
struct Triangle  { Vector2 a, b, c, center; };
```

Estas tres estructuras son las que usan las funciones de colisión y las primitivas de dibujado. Son agregados triviales: se construyen con llaves y se leen por campo.

20. Texto y fuentes

El motor dibuja texto a partir de fuentes horneadas: un PNG con los glifos y un JSON con sus métricas. Así se evita cargar un TTF en cada fotograma.

20.1 Horneear una fuente

ENG::BakerFont convierte un archivo TTF en la pareja PNG + JSON que el motor entiende. El proceso solo hay que hacerlo una vez, cuando cambias la fuente.

src/Game/Game.cpp (comentado en el proyecto)

```
// ENG::BakerFont baker("Font/TTF/Roboto.ttf");
// baker.Bake( {ENG::Language::English, ENG::Language::Spanish} );
// baker.SaveToDisk("assets/Font/", "Roboto");
```

Idiomas Se pueden hornear varios idiomas a la vez; cada uno genera su propio archivo JSON. Las fuentes incluidas en el proyecto son **PottaOne**, **Cabin** y **CabinItalic**.

20.2 Cargar y registrar una fuente

`src/Game/Game.cpp`

```
ENG::Services::Fonts().LoadFont(std::make_unique<ENG::Font>("PottaOne"));
ENG::Services::Fonts().LoadFont(std::make_unique<ENG::Font>("Cabin"));
ENG::Services::Fonts().LoadFont(std::make_unique<ENG::Font>("CabinItalic"));
```

Función	Devuelve	Notas
<code>LoadFont(std::unique_ptr)</code>	—	Toma posesión de la fuente.
<code>GetFont(name)</code>	<code>Font*</code>	Devuelve <code>nullptr</code> si no existe.
<code>Clear()</code>	—	Libera todas las fuentes.

20.3 La clase Font

Método	Devuelve	Para qué
<code>Font(name)</code>	—	Carga desde <code>assets/Font/<name>.json</code> y su PNG.
<code>GetName()</code>	<code>std::string</code>	Nombre de la fuente.
<code>GetTexture()</code>	<code>std::shared_ptr<RImage></code>	El PNG de los glifos.
<code>GetLineHeight()</code>	<code>float</code>	Altura de línea, útil para interlineado.
<code>GetGlyphs(int c)</code>	<code>const Glyph*</code>	Métricas de un carácter por su <code>*code point</code> .

20.4 Escribir texto

Hay dos formas de escribir texto. La primera es un objeto de la escena, la segunda es dibujar directamente en `OnRender()`.

Opción A: como objeto de la escena

```
// un objeto de texto vive en el pool y se dibuja con la cámara
ENG::Object* texto = pool.Get(idTexto);
texto->SetPosition(ENG::Vector2{100, 50});
```

Opción B: directamente en OnRender()

```

ENG::TextAPI& api = ENG::TextAPI::Get();
auto& font = *ENG::Services::Fonts().GetFont("Cabin");

api.DrawText(font, 100.0f, 50.0f, 24.0f, "Hola mundo");
api.DrawTextEx(font, ENG::Vector2{100, 90}, "Con color", 24.0f,
ENG::Color::Yellow);

float ancho = api.GetMeasureText(&font, "Hola mundo", 24.0f);

```

Función	Para qué
<code>DrawText(font, x, y, size, text)</code>	Dibujo rápido por coordenadas.
<code>DrawTextEx(font, pos, text, size, color)</code>	Dibujo con posición y color.
<code>FormatText(fmt, ...)</code>	Igual que <code>printf</code> , devuelve un <code>std::string</code> .
<code>GetMeasureText(font, text, size)</code>	Ancho en píxeles del texto.
<code>GetMeasureTextEx(font, text, size)</code>	Ancho y alto como <code>Vector2</code> .

Capa UI para el texto Si el texto debe verse siempre, dale `LAYER_UI` al objeto. Si es parte del mundo, la cámara 2D lo desplazará junto con el resto.

21. Interfaz de usuario

La GUI del motor son objetos como cualquier otro. Se crean, se añaden al pool y se dibujan en la misma pasada que el resto.

21.1 Los tres controles

Clase	Para qué	Métodos destacados
<code>ENG::Button</code>	Botón con textura, texto y acción.	<code>SetTexture</code> , <code>SetText</code> , <code>SetFunction</code> , <code>SetData</code> , <code>Action</code>
<code>ENG::Label</code>	Texto sobre un fondo opcional.	<code>SetText</code> , <code>SetColorText</code> , <code>SetFont</code> , <code>SetTextAlign</code> , <code>SetBackgroundColor</code>
<code>ENG::TextBox</code>	Campo de entrada de texto.	<code>GetTextStorage</code> , <code>SetDelimiter</code> , <code>SetText</code> , <code>SetFont</code>

21.2 Crear un botón

src/Engine/GUI/Button.hpp

```
Button(const std::string& _name, const Vector2& dimention);
Button(const std::string& _name, const Vector2& dimention,
std::function<int(std::any)> _func);

void SetTexture(const std::string& atlas, const std::string& textName);
void SetBackgroundColor(const Color& color);
void SetOutlineColor(const Color& color);
void SetFunction(ButtonCallback func);           // std::function<void(std::any)>
void SetData(std::any _);                       // el dato que recibiras al
pulsarlo
void SetText(const std::string& text);
void SetTextSize(float size);
void SetTextColor(const Color& color);
void SetFont(Font& font);
Rectangle GetRect(void);
void Action(void);                             // dispara el callback a mano
```

Uso

```
auto idBoton = pool.Add(std::make_unique<ENG::Button>("BotonJugar",
ENG::Vector2{200, 50}));

auto* boton = pool.Get(idBoton);
boton->SetBackgroundColor(ENG::Color::Blue);
boton->SetText("JUGAR");
boton->SetFont(*ENG::Services::Fonts().GetFont("Cabin"));
boton->SetTextSize(20.0f);
boton->SetData(std::string("nivel_01"));
boton->SetFunction([](std::any data)
{
    ENG::Services::Scenes().PeddingScene(std::string(data));
});
```

El truco de SetData Como el callback recibe un `std::any`, el mismo botón puede servir para varias acciones: guarda en `SetData()` el destino y recupéralo con `std::any_cast<...>()` dentro de la función.

21.3 Etiquetas y campos de texto

Label y TextBox

```
// etiqueta
auto id = pool.Add(std::make_unique<ENG::Label>("Marcador", ENG::Vector2{300, 40}));
auto* lbl = pool.Get(id);
lbl->SetText("Puntos: 0");
lbl->SetFontSize(22.0f);
lbl->SetColorText(ENG::Color::White);
lbl->SetTextAlign(ENG::AlignText::Center);           // Left, Center, Right

// campo de texto
auto idT = pool.Add(std::make_unique<ENG::TextBox>("Nombre", "",
ENG::Vector2{240, 40}));
auto* box = pool.Get(idT);
box->SetDelimiter('\n');
std::string escrito = box->GetTextStorage();
```

22. Audio: música y efectos

El audio usa SDL3_mixer. Hay dos gestores distintos: uno para música de fondo y otro para efectos, porque se portan de forma diferente.

22.1 Música

src/Game/Scenes/Demo.cpp

```
// opciones de SDL3: repetir en bucle infinito
SDL_PropertiesID options = SDL_CreateProperties();
SDL_SetNumberProperty(options, MIX_PROP_PLAY_LOOPS_NUMBER, -1);

ENG::Services::Music().LoadMusic("Music/MarineHoloLive.mp3", "Marine", options);
ENG::Services::Music().SetVolume("Marine", 10.0f);
ENG::Services::Music().PlayMusic("Marine");
```

Función	Qué hace
<code>LoadMusic(path, key, options)</code>	Carga y registra la pista.
<code>PlayMusic(key)</code>	Reproduce (o reanuda) la pista.
<code>StopMusic(key)</code>	Detiene la pista.
<code>SetVolume(key, volume)</code>	Fija el volumen.
<code>GetVolume(key)</code>	Devuelve el volumen actual.
<code>IsMusicLoaded(key)</code>	Comprueba si la pista está cargada.

Función	Qué hace
Clear()	Libera todo lo cargado.

22.2 Efectos de sonido

src/Game/Scenes/Demo.cpp

```
// el tercer argumento es cuantas copias del efecto se guardan en memoria
ENG::Services::SFX().LoadSFX("SFX/Shot_Gun.mp3", "GunShot", 2);
ENG::Services::SFX().SetVolume("GunShot", 10.0f);

// dispararlo cuando pasa algo
if (pollEvent.IsKeyPress(SDL_SCANCODE_F1))
    ENG::Services::SFX().PlaySFX("GunShot");
```

Qué es "el número de copias" Un efecto puede sonar varias veces a la vez. **LoadSFX** con **maxTracksMemory = 2** reserva dos pistas del mismo sonido; si se dispara una tercera vez mientras las dos suenan, se reutiliza la más antigua. La constante **MAX_TRACKS_MEMORY** vale 16.

22.3 Controlar el volumen con el teclado

src/Game/Scenes/Demo.cpp

```
if (pollEvent.IsKeyPress(SDL_SCANCODE_KP_PLUS))
{
    auto& m = ENG::Services::Music();
    float v = m.GetVolume("Marine") + 10.0f;
    m.SetVolume("Marine", v);
}

if (pollEvent.IsKeyPress(SDL_SCANCODE_KP_MINUS))
{
    auto& m = ENG::Services::Music();
    float v = m.GetVolume("Marine") - 10.0f;
    m.SetVolume("Marine", v);
}
```

23. Diálogos y CSV

El gestor de CSV lee tablas externas y sirve para diálogos y traducciones. Se apoya en rapidcsv, que viene incluida en el motor.

23.1 Cargar una tabla

src/Engine/Services/CSVManager.hpp

```
void Load(const std::string& path, const std::string& key, int colID = 0, int
rowID = 0);
rapidcsv::Document* GetDoc(const std::string& key);
```

El índice de la tabla **colID** y **rowID** son el índice de la columna y de la fila que identifican la tabla dentro del CSV: el motor guarda allí la clave con la que luego se busca cada entrada.

23.2 Leer un diálogo

src/Engine/Services/CSVManager.hpp

```
// una línea concreta
std::string linea = ENG::Services::CSV().GetText("Spanish", "intro",
"linea_01");

// un rango de líneas: de "linea_02" a "linea_05"
std::vector<std::string> rango =
    ENG::Services::CSV().GetRangeText("Spanish", "linea_02", "linea_05", "intro");
```

Parámetro	Significado
language	Columna del idioma: Spanish , English , etc.
dialog	Fila que agrupa las líneas de una conversación.
key	Clave con la que se cargó la tabla.
from / where	Primera y última línea del rango, inclusive.

23.3 El componente IDialog

IDialog es un componente de objeto: guarda el diálogo actual y lo escribe letra a letra en pantalla. Es lo que da el efecto de conversación clásica.

src/Engine/Component/Component.hpp

```
// por numero de linea
IDialog(const std::string& keyDialogs, int text);

// por identificadores de linea
IDialog(const std::string& keyDialogs, int fromText, int whereText);
IDialog(const std::string& keyDialogs, const std::string& dialog);
IDialog(const std::string& keyDialogs, const std::string& from, const
std::string& where);

void SetSpeed(float s);           // caracteres por fotograma
void SetStep(int step);
bool IsCompleteDialog(void) const;
```

Uso

```
obj->AddComponent<ENG::IDialog>("dialog", 1, 5);
obj->GetComponent<ENG::IDialog>()->SetSpeed(1.0f);

// en Update()
if (auto* dlg = obj->GetComponent<ENG::IDialog>())
{
    dlg->Update();
    if (dlg->IsCompleteDialog())
        ENG::Services::Scenes().PeddingScene("MainMenu");
}
```

IText no es lo mismo que **Text**. **IText** es un componente que guarda una cadena y un tamaño. La clase **ENG::Text** es un objeto completo con posición, fuente y dibujado. Usa **IDialog** si quieres el efecto de escritura progressive.

24. Colisiones y físicas

La física del motor es funcional, no basada en componentes: hay funciones sueltas que comparan cajas de colisión. Se combinan con **IBoundingBox**.

24.1 Las cajas de colisión

src/Engine/Object/Object.hpp y Component.hpp

```
// añadir una caja (el segundo argumento marca si es un trigger)
IBoundingBox& box = obj->AddBoundingBox(ENG::Vector2{32, 32}, false);

std::vector<IBoundingBox>& cajas = obj->GetBoundingBoxes();
box.Update(objeto.GetPosition()); // se actualiza cada fotograma con el
transform
ENG::Vector2 centro = box.GetCenter();
bool esTrigger = box.IsTrigger();
```

Método	Qué hace
AddBoundingBox(dim, isTrigger)	Añade una caja. Devuelve una referencia a ella.
GetBoundingBoxes()	Devuelve el vector de cajas.
Update(pos)	Mueve la caja. Lo hace ITransform cada fotograma.
ChangeSize(dim)	Cambia el tamaño.
GetSize() / GetPosition() / GetCenter()	Consultas de geometría.
IsTrigger() / SetTrigger(b)	Un trigger detecta pero no bloquea.

Las referencias pueden quedar obsoletas Las cajas viven en un `std::vector`. Si añades otra caja, las referencias y punteros a las anteriores pueden dejar de ser válidos. Usa índices o vuelve a pedir la caja tras cada alta.

24.2 Funciones de colisión

src/Engine/Physics/Collision.hpp

```
// entre objetos: con resolucion o solo deteccion
bool CollisionAABB(Object& source, Object& target, bool resolution);
bool CollisionAABB(const IBoundingBox& source, const IBoundingBox& target);
bool CollisionAABB(const Rectangle& a, const Rectangle& b);

// otras formas
bool CollisionPointRect(const Vector2& point, const Rectangle& rect);
bool CollisionCircle(const Circle& a, const Circle& b);
```

Ejemplo de colisión en una escena

```
#include "Engine/Physics/Collision.hpp"

void DemoScene::Update(float dt)
{
    auto& ids = pool.GetAllIDs();

    for (auto a : ids)
    {
        auto* objA = pool.Get(a);
        if (objA->GetName() != "Player") continue;

        for (auto b : ids)
        {
            if (a == b) continue;
            auto* objB = pool.Get(b);

            if (ENG::CollisionAABB(*objA, *objB, false))
            {
                // se tocan: dispara un efecto, cambia una variable, lo que necesites
                ENG::Services::SFX().PlaySFX("GunShot");
            }
        }
    }
}
```

Colisión por pares La función compara **dos** objetos. Detectar colisiones entre todos los pares de una escena es trabajo tuyo: recorre la lista de identificadores del pool y compara en dos bucles.

Triggers para zonas Una caja marcada como trigger detecta el solapamiento sin bloquear el movimiento. Es la forma cómoda de detectar zonas de diálogo, pickups o zonas de daño.

25. El tiempo: dt y el reloj

Todo el motor trabaja en segundos, no en fotogramas. El **ClockManager** mide el tiempo transcurrido y te lo entrega como **dt** en cada fase del bucle.

25.1 Cómo se mide el tiempo

src/Engine/Engine.cpp (el reloj se usa así)

```
auto clock = Services::Clock();
while (m_game->IsRunning())
{
    clock.Tick();
    auto dt = clock.GetDT();
    // ... dt se pasa a OnUpdateFixed, OnInputs, OnUpdate y OnRender ...
}
```

Función	Devuelve	Para qué
Init()	—	La llama ProvideClockSaver() . Pone el reloj a cero.
Tick()	—	Marca el fotograma actual y calcula dt .
GetDT()	float	Segundos desde el fotograma anterior.
GetTime()	float	Segundos desde el arranque.
Clear()	—	Libera el reloj.

El límite de 0,05 segundos **Tick()** recorta el tiempo a un máximo de 0,05 s. Si el juego se congela un instante (por ejemplo, al cambiar de escena), el fotograma siguiente no dará un salto gigante. Gracias a esto un objeto que se mueva a **300 px/s** nunca saltará varios cientos de píxeles de golpe.

25.2 Mover usando dt

Multiplicar siempre por **dt** es la diferencia entre un movimiento fluido y uno dependiente de los fotogramas por segundo.

src/Game/Systems/Systems.cpp (íntegro)

```
void System_PlayerMovement(ENG::Object& o, float dt)
{
    auto& pPos = o.GetTransform();
    pPos.m_direction.Normalize(); // la
    diagonal no corre mas rapido

    pPos.m_position += pPos.m_direction * pPos.m_velocity * dt; //
    independiente de los FPS
}
```

src/Game/Scenes/Demo.cpp (uso del sistema)

```
void DemoScene::Update(float dt)
{
    for (auto& o : pool.GetAllIDs())
    {
        auto obj = pool.Get(o);
        if (obj->GetName() == "Player")
            System_PlayerMovement(*obj, dt);

        obj->Update(dt);
    }
}
```

Mal	Bien	Por qué
<code>pos.x += 5;</code>	<code>pos.x += 5.0f * dt;</code>	A 30 FPS va a la mitad de velocidad que a 60.
<code>anim.Advance(1.0f);</code>	<code>anim.Advance(dt);</code>	La animación va a distinta velocidad.
<code>if (dir == 0) skip;</code>	normalizar <code>dir</code> primero	La diagonal sería más rápida que los ejes.

Antes y después Coloca la lógica en **Update(dt)** para la parte visual y en **UpdateFixed(dt)** para la física. La fase fija se ejecuta antes que la entrada y que la lógica, así que las colisiones se resuelven con un estado consistente.

26. Rutas y configuración

El motor resuelve todas las rutas **relativas al ejecutable**, no al directorio de trabajo. Esto permite que el juego se ejecute desde cualquier sitio sin romper los assets.

26.1 ENG::Path

src/Engine/Utils/Path.hpp

```
class Path
{
public:
    static Path& Get(void);
    static void Destroy();

    std::string ReadFileString(const std::filesystem::path& path) const;
    std::stringstream ReadFile(const std::filesystem::path& path) const;
    std::filesystem::path GetExecutableDir() const;
    const std::filesystem::path& ExecutableDir() const { return
m_executableDir; }

    // carpetas del proyecto
    std::filesystem::path AssetsPath;
    std::filesystem::path ShadersPath;
    std::filesystem::path ConfigPath;
};
```

Nunca asumas la ruta de trabajo Si ejecutas el binario desde otro directorio, las rutas relativas normales fallarían. Usa siempre `ENG::Path::Get().AssetsPath / "mi/archivo.png"`.

26.2 Qué se copia al compilar

CMake copia estas carpetas junto al binario en cada compilación. Si añades una carpeta nueva, hay que registrarla en el `CMakeLists.txt` correspondiente.

Carpeta	Contiene	Ruta en tiempo de ejecución
<code>assets/</code>	Atlas, diálogos, entidades, fuentes, iconos, mapas, música y efectos.	junto al ejecutable
<code>shaders/</code>	Archivos <code>.vs</code> y <code>.fs</code> .	junto al ejecutable
<code>config/</code>	<code>eng.json</code> .	junto al ejecutable

26.3 El archivo de configuración

config/eng.json

```
{
  "name": "Project_AK",
  "screen": {
    "w": 1920,
    "h": 1080
  },
  "fullscreen": true
}
```

`main()` lo lee al arrancar y lo **vuelve a escribir al salir**, así que si juegas en pantalla completa y luego cierras, la próxima vez abrirá en pantalla completa.

src/Engine/Utils/Config.hpp

```

class EngineConfig
{
public:
    nlohmann::json Save()
    {
        return {
            { "name", m_title },
            { "screen", {{ "w", m_viewWidth }, { "h", m_viewHeight }} },
            { "fullscreen", m_eFullscreen }
        };
    }

    void Load(const nlohmann::json& j)
    {
        m_title = j.value("name", m_title);
        if (auto it = j.find("screen"); it != j.end() && it->is_object())
        {
            m_viewWidth = it->value("w", m_viewWidth);
            m_viewHeight = it->value("h", m_viewHeight);
        }
        m_eFullscreen = j.value("fullscreen", m_eFullscreen);
    }

    std::string m_title = "";
    unsigned int m_viewWidth = 0;
    unsigned int m_viewHeight = 0;
    bool m_eFullscreen;
};

```

Si falta el archivo Si **eng.json** no existe, **main()** avisa por log y arranca con los valores por defecto: 1920×1080, ventana y título **Project_AK**. No se cae.

26.4 Guardar el estado de la ventana

src/Engine/Engine.cpp

```

void Engine::OnSave(void)
{
    auto& r = Render::Get();
    this->m_eConfig->m_eFullscreen = r.GetFullscreen();
    this->m_eConfig->m_viewWidth = r.GetScreenSize().x;
    this->m_eConfig->m_viewHeight = r.GetScreenSize().y;
}

```

27. Registro de mensajes (logging)

El motor no usa **cout** ni **printf**: usa sus propias macros, que añaden marca de tiempo, archivo y número de línea automáticamente.

27.1 Las cuatro macros

src/Engine/Utils/Log.hpp

```
#define LOG_INFO(msg)  ENG::Logger::Get().Log(ENG::LogLevel::INFO, msg,
std::source_location::current())
#define LOG_DEBUG(msg) ENG::Logger::Get().Log(ENG::LogLevel::DEBUG, msg,
std::source_location::current())
#define LOG_ERROR(msg) ENG::Logger::Get().Log(ENG::LogLevel::ERROR, msg,
std::source_location::current())
#define LOG_FATAL(msg) ENG::Logger::Get().Log(ENG::LogLevel::FATAL, msg,
std::source_location::current())
```

Macro	Cuándo usarla	Qué imprime
LOG_INFO	Mensajes de progreso: "ventana creada", "escena cargada".	Marca de tiempo y mensaje.
LOG_DEBUG	Trazas de depuración. Se desactivan en compilación de publicación.	Añade archivo y línea.
LOG_ERROR	Algo falló pero el programa puede seguir.	Añade archivo y línea.
LOG_FATAL	Algo falló y no se puede continuar.	Añade archivo, línea y función.

Uso

```
#include "Engine/Utils/Log.hpp"

LOG_INFO(" | << Game application Init Succesfully");

if (!atlas)
    LOG_ERROR(" | << ATLAS not found: " + atlasKey);

if (!SDL_Init(0))
    LOG_FATAL(" | << SDL could not be initialised");
```

Las cadenas se concatenan con + El parámetro es un **std::string**, no un **printf**. Si mezclas un literal con un **std::string**, el + de C++ no admite el primer operando como cadena en C++17: envuelve el literal o construye la cadena completa antes.

Fíjate en el prefijo Los mensajes del motor empiezan por **| <<**. Es una convención de la casa: mantenla en tus propios mensajes para que la consola se lea igual.

28. Guardar y cargar el mundo

WorldSaver guarda el estado de los objetos en memoria y lo serializa a JSON en disco. Es la base de cualquier sistema de guardado de partida.

28.1 La API

Función	Qué hace
<code>SaveObject(const ObjectState&)</code>	Guarda (o reemplaza) el estado de un objeto en memoria.
<code>GetObject(const std::string& key)</code>	Devuelve el estado guardado o <code>nullptr</code> .
<code>HasObject(key)</code>	Comprueba si existe.
<code>RemoveObject(key)</code>	Borra una entrada.
<code>SaveToDisk(path)</code>	Vuelca todo a un archivo JSON. Devuelve <code>bool</code> .
<code>LoadFromDisk(path)</code>	Lee el archivo. Devuelve <code>bool</code> .
<code>Clear()</code>	Vacía la memoria.

28.2 Guardar y recuperar un objeto

`src/Engine/Object/Object.hpp` y `WorldSaver.hpp`

```
class Object
{
public:
    ObjectState Save() override;           // serializa este objeto
    void Load(const ObjectState& state) override; // lo restaura
};

// uso
auto* obj = pool.Get(idPlayer);
ENG::Services::WSaver().SaveObject(obj->Save());

ENG::Services::WSaver().SaveToDisk("World/partida_01.json");
```

28.3 Serialización con nlohmann/json

`WorldSaver` usa las funciones `to_json` y `from_json` de `nlohmann/json`. Si añades un campo a `ObjectState`, actualiza también esas dos funciones, o el campo no se guardará.

`src/Engine/Services/WorldSaver.hpp` (extracto)

```
struct ObjectState
{
    std::string name;
    Vector2      position;
    uint8_t      layer;
    std::string shaderKey;
};

void to_json(nlohmann::json& j, const ObjectState& s);
void from_json(const nlohmann::json& j, ObjectState& s);
```

Copia el estado, no el objeto `SaveObject()` recibe un `ObjectState`, no un `Object*`. El estado se guarda **por valor**: si el jugador se mueve después de guardar, el registro no cambia. Vuelve a llamar a `Save()` para actualizarlo.

Estructura recomendada Guarda la partida al salir de la escena o al pulsar una combinación de teclas, y carga al arrancar antes de crear la escena. Un `ObjectState` por objeto, todos en un único JSON, es más fácil de depurar que un archivo por objeto.

PARTE III · REFERENCIA

29. Mapa del proyecto

Saber dónde está cada cosa es la mitad del trabajo. Este es el mapa completo de las carpetas del repositorio y qué hay dentro de cada una.

29.1 Raíz del repositorio

Ruta	Qué contiene
<code>assets/</code>	Recursos del juego. Se copia junto al binario al compilar.
<code>shaders/</code>	Shaders GLSL (<code>mono</code> y <code>gl/</code>). También se copia al compilar.
<code>config/</code>	<code>eng.json</code> , la configuración de la ventana.
<code>src/</code>	Todo el código.
<code>CMakeLists.txt</code>	Configuración de la compilación y las dependencias.
<code>runner.sh</code>	Atajo: prepara <code>build/</code> , compila en Debug y ejecuta.
<code>CLAUDE.md</code> , <code>AGENTS.md</code>	Convenciones del proyecto para quien lo maintains.

29.2 Dentro de src/

Carpeta	Qué contiene
<code>src/Main/</code>	<code>main.cpp</code> : lee la configuración, crea el Engine y ejecuta el ciclo de vida.
<code>src/Engine/</code>	El motor. Es reutilizable: no sabe nada de tu juego.
<code>src/Game/</code>	Tu juego: <code>Game.hpp/cpp</code> , <code>Scenes/</code> y <code>Systems/</code> .

29.3 Dentro de src/Engine/

Carpeta	Responsabilidad
<code>Component/</code>	La clase IComponents y todos los componentes, más ComponentStorage .
<code>Extern/</code>	STB, rapidcsv y otras bibliotecas de terceros.
<code>GUI/</code>	Button, Label y TextBox.
<code>Inputs/</code>	PollEvent y Mouse.
<code>Layer/</code>	Scene y GameLayer: las dos clases base que heredas.

Carpeta	Responsabilidad
Map/TileMap/	Carga y dibujado de mapas de Tiled.
Modules/	BakerFont y los rangos de idioma.
Object/	Object, ObjectPool y RenderEntry.
Physics/	Las funciones de colisión.
Render/	Todo el dibujado: lote, cámara, color, geometría, imágenes, shaders y luces.
Services/	Los nueve managers y el localizador Services .
Text/	Text, TextAPI y las fuentes.
Utils/	Vector2, Color, Log, Path, Config y geometría simple.

29.4 Dentro de src/Game/

Archivo	Responsabilidad
Game.hpp / Game.cpp	La capa del juego. Carga los recursos globales y delega en la escena activa.
Scenes/Demo.hpp / .cpp	La escena de demostración: es el ejemplo más completo.
Scenes/Plantilla.hpp	Plantilla vacía para copiar al crear una escena nueva.
Systems/Systems.hpp / .cpp	Funciones de lógica separadas de la escena (movimiento, IA, etc.).

De dónde copiar Para crear una escena nueva, copia **Scenes/Plantilla.hpp** y sigue la estructura de **Scenes/Demo.cpp**. Recuerda añadir el nuevo **.cpp** al **CMakeLists.txt** de **Scenes/**.

CMake es explícito Un **.cpp** nuevo no se compila solo. Hay que añadirlo con **target_sources(Project_AK PRIVATE ...)** en el **CMakeLists.txt** de su carpeta, y si la carpeta es nueva, añadir **add_subdirectory()** en el de la carpeta padre. Después, regenera desde **build/**.

30. Referencia rápida: "¿cómo hago X?"

La mayoría de las dudas del día a día se resuelven con una de estas recetas.

Quiero...	Lo que debo escribir
Crear un objeto en una	<code>auto id = pool.Add(std::make_unique<ENG::Object>("Nombre"));</code>

Quiero...	Lo que debo escribir
escena	
Ponerle una textura	<code>pool.Get(id)->AddComponent<ENG::ISprite>("ClaveTextura");</code>
Animar un objeto	<code>AddComponent<ENG::IAnimator>("ClaveAtlas", frames, fps);</code> y luego <code>->Play()</code>
Tintar un objeto	<code>AddComponent<ENG::IColor>(ENG::Color::Red);</code>
Darle un shader	<code>AddComponent<ENG::IMaterial>("Mono");</code>
Hacerlo colisionable	<code>obj->AddBoundingBox(ENG::Vector2{32, 32});</code>
Cambiar su posición	<code>obj->SetPosition(ENG::Vector2{x, y});</code>
Cambiar su capa	<code>obj->SetLayer(LAYER_PLAYER);</code>
Moverlo cada fotograma	<code>obj->Update(dt);</code> sobrescrito, usando <code>m_position += dir * vel * dt;</code>
Cambiar de escena	<code>isRunning = false; + Services::Scenes().PeddingScene("Otra");</code>
Cargar una textura	<code>Services::Assets().Load("ruta.png", "Clave");</code>
Cargar un atlas	<code>Services::Assets().LoadAtlas("atlas.json", "Clave");</code>
Cargar un mapa de Tiled	<code>for (auto& m : TileMap::LoadTiledMap("ClaveAtlas", "mapa.json")) AddTileMap(std::move(m));</code>
Cargar un shader	<code>Services::Shaders().Load("mono.fs", "mono.vs", "Mono");</code>
Cargar una fuente	<code>Services::Fonts().LoadFont(std::make_unique<ENG::Font>("Cabin"));</code>
Poner música	<code>Services::Music().LoadMusic("musica.mp3", "Clave");</code> y <code>PlayMusic("Clave");</code>
Tocar un efecto	<code>Services::SFX().LoadSFX("sonido.mp3", "Clave", 2);</code> y <code>PlaySFX("Clave");</code>
Escribir texto	<code>TextAPI::Get().DrawText(font, x, y, size, "Hola");</code>
Dibujar un rectángulo	<code>ENG::Drawer::DrawRectangle({x, y, w, h}, ENG::Color::Red);</code>

Quiero...	Lo que debo escribir
Detectar una tecla	<code>PollEvent::Get().IsKeyPress(SDL_SCANCODE_P);</code>
Mover la cámara	<code>cam->SetTarget(&obj->GetTransform()); + GetBatcher().SetCamera2D(cam.get());</code>
Hacer zoom	<code>cam->SetZoom(std::clamp(zoom, 1.0f, 4.0f));</code>
Abrir mensaje en consola	<code>LOG_INFO(" << mensaje");</code>
Guardar la partida	<code>Services::WSaver().SaveObject(obj->Save()); y SaveToDisk("ruta.json");</code>
Cerrar el juego	<code>m_gameRunning = false; en Game::IsRunning()</code>

El orden importa en tres sitios 1. Los **shaders** se cargan antes que las escenas, porque las escenas los usan al construirse. 2. El **atlas** se carga antes que el mapa, porque `LoadTiledMap()` busca el atlas por clave. 3. La **fuentes** se carga antes de usarla en un **Label** o un **Button**.

31. Errores comunes

Casi todos los errores del primer contacto con el motor tienen la misma forma: un mensaje de compilación, una pantalla en negro o un comportamiento raro. Esta es la lista de los que más se repiten.

31.1 Errores de compilación

El archivo nuevo no se compila

Síntoma: creaste un `.cpp` nuevo y el enlazador se queja de símbolos sin definir, o simplemente nada de lo que escribiste hace falta.

CMakeLists.txt de Scenes/

```
target_sources(Project_AK PRIVATE
    Demo.cpp
    MainMenu.cpp
    MiEscenaNueva.cpp      # <- tu archivo nuevo
)
```

Si además creaste una carpeta nueva, hay que registrarla en el `CMakeLists.txt` de la carpeta padre. Después, vuelve a generar desde `build/`:

Regenerar el proyecto

```
cd build
cmake -G "Ninja Multi-Config" ..
cmake --build . --config Debug --target Project_AK
```

Los includes no se resuelven

Síntoma: `fatal error: Game/Game.hpp: No such file or directory`

La forma correcta de incluir

```
#include "Engine/Render/Render.hpp"    // correcto: relativo a src/
#include "Game/Game.hpp"                // correcto
#include "../../../Engine/Render/Render.hpp" // NO: rutas relativas, frágil
```

Sin --config no compila El proyecto usa **Ninja Multi-Config**, que no es un generador de configuración única. Si lanzas `cmake --build build` sin `--config`, el error es **Error: could not load cache** o similar. Usa siempre `--config Debug` o `--config Release`.

El atributo se repite

Síntoma: `Layout location ... already used` al compilar un shader.

shaders/gl/texture.vs

```
layout(location = 0) in vec3  a_Position;
layout(location = 1) in vec4  a_Color;
layout(location = 2) in vec2  a_TexCoord;
layout(location = 3) in float a_TexIndex;
layout(location = 4) in float a_ArrayLayer;
layout(location = 5) in float a_EffectMode;
```

Los layouts son 1 a 1 Cada atributo necesita un **location** distinto en el vértice y en el fragmento, y el número debe coincidir con el que espera el motor. Si añades un atributo, resérvalo al final de la lista y no cambies los números existentes.

Falta la implementación de una biblioteca

Síntoma: `undefined reference to 'glCreateProgram'` o similar.

Casi siempre es que el GLAD no se ha implementado en ningún punto del programa. La implementación va en `src/Main/main.cpp`, junto a las de STB.

src/Main/main.cpp

```
#define STB_IMAGE_IMPLEMENTATION
#include "Engine/Extern/stb_image.h"
#define STB_IMAGE_WRITE_IMPLEMENTATION
#include "Engine/Extern/stb_image_write.h"
#define STB_TRUETYPE_IMPLEMENTATION
#include "Engine/Extern/stb_truetype.h"
```

Si la cadena literal es la que falla, envuélvela con `std::string(...)` o concatena con `std::string`: el `+` de C++ exige que el primer operando ya sea una cadena. En el motor todos los mensajes empiezan por `| <<` y llevan archivo y línea. Cuando algo falle, lee el log completo antes de tocar el código: casi siempre el error ya está escrito ahí.

31.2 Errores de ejecución

Pantalla en negro

- El shader no se cargó: revisa el log. Si el nombre está mal, la pantalla se queda en el color de limpieza.

- La cámara no está conectada al **Batcher**: sin **SetCamera2D()** no hay matriz vista-proyección válida.
- El objeto está en una capa que no se dibuja en ese tramo de **Render()**.
- El atlas no se cargó, así que las UV apuntan a una textura vacía.

El mapa sale desplazado

LoadTiledMap() coloca cada mapa en **{500, 500}** al cargarlo. Si tu mapa aparece fuera de sitio, corrige la posición con **TileMap::SetPosition()** o ajusta la cámara.

Se ve la textura equivocada

El lote tiene 16 ranuras de textura y la 0 la ocupa una textura blanca fija. Si dibujas más de 15 texturas distintas en un fotograma, el motor reutiliza la última ranura. Agrupa los recursos en atlas.

Las teclas no responden

- Estás consultando en **OnRender()** en lugar de **OnInputs()**. Los eventos se borran al final del fotograma.
- Usas **IsKeyPress** para algo continuo. Para moverse necesitas **IsKeyDown**.
- No has llamado a **ENG::Services::Scenes().GetCurrent()->Inputs(dt)** desde **Game::OnInputs()**.

El cambio de escena no ocurre

- Has puesto **isRunning = false** pero no has llamado a **PeddingScene("Id")**. Sin destino, **ChangeScene()** no tiene a dónde ir.
- El **SceneID** que pasas a **PeddingScene()** no coincide con el que diste en **AddScene()**. Son cadenas: distingue mayúsculas.
- La escena nueva necesita su propia cámara conectada en su **Init()**.

El juego va más rápido o más lento según el equipo

Estás moviendo con constantes fijas en vez de multiplicar por **dt**. En un equipo con 144 Hz el juego irá más rápido que en uno con 60 Hz.

Los objetos se dibujan en orden raro

La cola de dibujado se ordena una sola vez, cuando llamas a **Sort()**. Si añades o quitas objetos después, hay que volver a llamar a **pool.Sort()** y copiar el resultado a **renderQueue**.

Errores de compilación y depuración

Los mensajes del motor empiezan por **| <<** y llevan archivo y línea. Cuando algo falle, lee el log completo antes de tocar el código: casi siempre el error ya está escrito ahí.

31.3 Errores de diseño

Lo que haces	Lo que conviene hacer
auto obj = new ENG::Object("X");	pool.Add(std::make_unique<ENG::Object>("X")); — el pool es el dueño.

Lo que haces	Lo que conviene hacer
<code>GetComponent<ISprite>()</code> sin comprobar	Comprueba el puntero: puede ser <code>nullptr</code> .
Guardar la referencia de una caja y seguir añadiendo cajas	Vuelve a pedir la caja: el <code>vector</code> puede reubicarse.
Cargar la misma textura muchas veces	Carga una vez y reutiliza la clave.
Un <code>ObjectPool</code> local dentro de una función	Como miembro de la escena: debe vivir con ella.
Crear objetos cada fotograma	Créalos en <code>Init()</code> y muévelos en <code>Update()</code> .
Mezclar <code>SDL3_mixer</code> sin pasar por <code>Services</code>	Usa <code>Services::Music()</code> y <code>Services::SFX()</code> .
Ignorar <code>Destroy()</code> al cambiar de escena	Libera ahí lo que creaste: el pool, los <code>TileMaps</code> , la cámara.

32. Glosario

Términos que aparecen en el código y en esta guía, con su significado en el contexto de Project AK.

Término	Significado
Engine	El motor gráfico: todo lo que hay en <code>src/Engine/</code> . No conoce tu juego.
GameLayer	La capa que implementa tu juego. Hereda de <code>ENG::GameLayer</code> y el motor la ejecuta.
Scene	Una pantalla del juego, con su propio contenido, su <code>ObjectPool</code> y su cámara.
SceneID	El identificador de texto con el que una escena se registra y se localiza.
Object	Cualquier entidad de la escena: jugador, botón, texto, enemigo.
ObjectPool	El dueño de los objetos de una escena. Da identificadores y orden de dibujado.
ObjectID	El <code>uint32_t</code> que devuelve <code>ObjectPool::Add()</code> . <code>NULL_ID</code> es <code>0</code> y no es válido.
Component / IComponents	Pieza de comportamiento que se engancha a un objeto.
ComponentStorage	El mapa interno que guarda los componentes de un objeto por tipo.
ITransform	Componente con posición, dirección, velocidad y posición anterior.
IBoundingBox	Caja de colisión de un objeto. Puede ser un trigger.

Término	Significado
Batcher	El lote de dibujado: acumula geometría del fotograma y la envía de una vez.
RenderEntry	Una entrada de la cola de dibujado: identificador, capa y shader.
Drawer	Fachada con las primitivas de dibujado (rectángulo, círculo, línea, polígono).
Camera2D	Produce la matriz vista-proyección que aplica el Batcher .
Atlas / AtlasData	Una imagen con muchas imágenes dentro, descrita por un JSON.
TextureArray	Array de texturas 2D de OpenGL donde el motor guarda los atlas.
UVRect	Rectángulo de coordenadas de textura: uvMin , uvMax y tamaño en píxeles.
Shader / Material	Programa de la GPU. El "material" de un objeto es su shader asignado.
TileMap	Mapa de cuadrículas que dibuja celdas de un atlas.
GID	El identificador numérico de celda del JSON de Tiled. El 0 es celda vacía.
Service / Services	Localizador de servicios: acceso global a los managers del motor.
Manager	Cada módulo con estado propio: assets, escenas, shaders, fuentes, audio, reloj, CSV.
BakerFont	Herramienta que convierte un TTF en el PNG + JSON que usa el motor.
Glyph	Las métricas y UV de un carácter dentro de una fuente horneada.
WorldSaver	Servicio que serializa el estado de los objetos a JSON.
ObjectState	Copia serializable del estado de un objeto.
dt	Segundos transcurridos desde el fotograma anterior, limitado a 0,05.
Frame / fotograma	Una vuelta completa del bucle principal.
Draw call	Una llamada de dibujado a la GPU. El Batcher las reduce al mínimo.
Trigger	Caja de colisión que detecta solapamientos sin bloquear el movimiento.
Path	El servicio de rutas. Resuelve todo respecto al ejecutable .
Config / EngineConfig	Los datos de config/eng.json : título, tamaño y pantalla completa.
LOG_INFO / LOG_ERROR / ...	Las cuatro macros de mensaje del motor.
CMake	El sistema de compilación. Los .cpp nuevos hay que registrarlos a mano.